

BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION COMPUTER SCIENCE

DIPLOMA THESIS

Enhancing OCR Performance through Image Super-Resolution

Supervisor
Asist. Drd. Horea Mureșan

Author
Bogdan-Tudor Vlad

2025

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

LUCRARE DE LICENȚĂ

Îmbunătățirea OCR prin intermediul super-rezoluției imaginilor

Conducător științific
Asist. Drd. Horea Mureșan

Absolvent
Bogdan-Tudor Vlad

2025

ABSTRACT

This thesis investigates the use of Image Super-Resolution (ISR) to enhance the performance of Optical Character Recognition (OCR) systems. OCR is an important computer vision task that involves the automatic recognition of printed or handwritten text in images. However, the accuracy of OCR systems can be limited by the resolution of the input images, which can be particularly low for images captured in real-world settings. To address this issue, this thesis explores the use of deep learning-based ISR (image super-resolution) techniques to enhance the resolution of low-resolution OCR images. The proposed approach involves using a neural network to learn the mapping between low-resolution and high-resolution image pairs. The network is used to generate high-resolution versions of low-resolution OCR images, which are subsequently fed into an OCR system.

Experimental results demonstrate that the proposed approach can significantly improve OCR accuracy on some low-resolution image dataset. In particular, the use of ISR techniques on low resolution images can bring the performance of an OCR system within 1% of the performance on the original high-resolution images for scanned documents and for certain OCR competitions specialized in text recognition on already cropped text detections. However, it worth noting that significant OCR performance improvements have not been found on real-world scene text images. We hypothesize that often times when there is enough information in an image to accurately reproduce the individual characters within it that information is also enough for achieving satisfactory OCR performance.

Overall, this thesis provides insights into the potential benefits of using ISR techniques to enhance OCR performance and the shortcoming which come with using such a pre-processing step. The proposed approach represents a promising direction for improving the accuracy of OCR systems, particularly for images of scanned documents such as books, product labels, legal contracts, or historical documents.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objective	2
1.3	Structure	2
2	Related works	4
2.1	General methods for Image Super-Resolution	4
2.2	Learning based image super-resolution	5
2.2.1	CNNs	5
2.2.2	ResNets	6
2.2.3	Perceptual loss	6
2.2.4	GANs	7
2.2.5	Transformers	8
3	Optical Character Recognition	9
3.1	Introduction	9
3.2	The link between OCR and SISR	10
4	SISR - Discussion	12
4.1	Datasets	12
4.2	Degradation methods	12
4.3	SISR metrics and their shortcomings	14
5	Experiment - Proof of concept	16
5.1	POC idea	16
5.2	Attempts to simple OCR performance boosts	17
5.3	Architecture choice	17
5.4	OCR improvements achieved through SISR	21
6	Further Investigations	24
6.1	The need for additional experiments	24
6.2	The FUNSD dataset	25

6.3	Tile SISR inference	26
6.4	How to match bounding boxes	27
6.5	The good, the bad, and the slow	29
6.5.1	The good	29
6.5.2	The bad	31
6.5.3	The slow	31
6.6	The Total Text dataset	33
6.6.1	Shortcomings of Google's Tesseract OCR	34
6.6.2	Original vs. SISR	36
6.7	Low Resolution Scanned Text Dataset	37
6.7.1	The experiment	38
6.8	Incidental Scene Text Dataset	40
6.9	Comparison with SOTA	40
6.10	Pre-trained models vs. fine-tuning vs. training from scratch	41
7	Tools and technologies	45
7.1	Backend	45
7.1.1	Python	45
7.1.2	PyTorch	46
7.1.3	Flask	46
7.1.4	Matplotlib	46
7.1.5	OpenCV	47
7.2	Frontend	47
7.2.1	React	47
8	Application Implementation	48
8.1	Description	48
8.2	Requirements and specification	48
8.2.1	Functionalities	48
8.2.2	Performance	49
8.2.3	Portability	50
8.3	Application design	50
8.3.1	Server architecture	51
8.3.2	Client architecture	53
8.3.3	Use cases	54
8.4	Application flow	54
8.5	Deployment	57
8.6	Testing	57
8.6.1	Unit testing	57
8.6.2	Integration testing	58

9	Conclusions and future work	59
9.1	Achievements	59
9.2	Limitations	59
9.3	Future improvements	60
9.4	Conclusions	60
	Bibliography	62

Chapter 1

Introduction

1.1 Motivation

Optical Character Recognition (OCR) is a fundamental computer vision task that involves the automatic recognition of printed or handwritten text in images. OCR has a wide range of applications, including document digitization, text recognition, and handwriting analysis. However, the accuracy of OCR systems can be limited by several factors, including the quality of the input images, variations in font type and size, and background noise.

One of the primary factors that limit OCR accuracy is the resolution of the input images. Low-resolution images, which are common in real-world settings, can be particularly challenging for OCR systems, as they may not contain enough detail to accurately identify individual characters. This is particularly problematic for handwritten text, which often contains subtle variations in stroke width and direction that can be difficult to capture in low-resolution images.

To address this issue, Image Super-Resolution (ISR) techniques have been proposed as a way to enhance the resolution of low-resolution images. ISR techniques involve using deep learning-based approaches to generate high-resolution versions of low-resolution images. The generated high-resolution images can then be used to improve the accuracy of OCR systems by providing additional details that may be missing from the low-resolution input images.

Motivated by the potential benefits of using ISR techniques for OCR, this thesis aims to investigate the use of deep learning-based ISR techniques to enhance OCR performance on low-resolution images. The proposed approach involves training a neural network to learn the mapping between low-resolution and high-resolution image pairs, which is then used to generate high-resolution versions of low-resolution OCR images. The resulting high-resolution images are then fed into an OCR system, and the accuracy of the OCR system is evaluated on the generated images.

1.2 Objective

The primary objective of this thesis is to investigate the potential of using image super-resolution (ISR) techniques to enhance the performance of Optical Character Recognition (OCR) systems. Specifically, the objective is to evaluate the effectiveness of state-of-the-art ISR methods, including both traditional and deep learning-based approaches, in improving the accuracy of OCR systems when working with low-resolution images.

To achieve this objective, the thesis will explore the current state of the art in ISR and OCR technologies, including the underlying algorithms and techniques used in these fields. The thesis will also evaluate the performance of several existing ISR and OCR systems, both individually and in combination, using a set of standard benchmark datasets.

1.3 Structure

The paper is divided in 9 chapters: **Related works, Optical Character Recognition, SISR - Discussion, Experiment - Proof of concept, Further investigations, Tools and technologies, Application implementation, and Conclusions and future work**. Each chapter focuses on different aspects of this topic.

First, we go through a quick history lesson on Image Super-Resolution (ISR). Here we look at how the advancements of Convolutional Neural Networks (CNNs)[1] and Generative Adversarial Networks (GANs)[2] helped us move away from classical Image Super-Resolution techniques such as bicubic interpolation[3] or patch-based approaches[4] and towards faster and better methods. Also, we explore how besides the large popularity of GANs in the field of Image Super-Resolution, we can observe how in recent years Transformers[5] have been used to get great performance while significantly reducing the number of parameters and computation.[6, 7]

Secondly, we take a quick look at what is Optical Character Recognition and how SISR can help improve such a task. We show that one significant factor that can limit OCR is the low resolution or high amount of noise of the input images which come from real-world applications. If we were to solve these issues then, intuitively, we would expect other machine learning models placed further down the pipeline to have improved performance.

Thirdly, we go more in depth into Single Image Super-Resolution. We look at what researchers have done and used over the years in terms of datasets and degradation methods. We also look at how SISR systems are evaluated through various metrics and why these metrics are imperfect and often times produce counter-

intuitive results.

Fourthly, we jump straight to our first experiment. We use a simple dataset from a text image super-resolution competition as a proof of concept to see if our approach has any chance of working. A satisfactory result on this dataset would provide the evidence we need to justify our idea and carry out the rest of our experiments.

Then, we carry out the rest of our experiments. We use four different and varied datasets to get the most complete view possible of our hypothesis. We use our super-resolution pre-processing step in images ranging from scanned documents to real-world scene text. We study where our approach improves OCR performance and where it doesn't, how much this additional pre-processing step slow down processing time, and how our method compares to SOTA.

In the next part we discuss the tools and technologies we used during our experiments and during application development.

Then, we describe our application and show how it works and how a user can navigate it. We talk about the design of the application, its functionalities and flow, and how we tested it.

Finally, we conclude this thesis by talking about the results of our method, limitations, and possible future improvements.

Chapter 2

Related works

2.1 General methods for Image Super-Resolution

The challenge of image super-resolution reconstruction is a complex mathematical problem that remains unresolved. Obtaining a unique high-resolution image from a low-resolution image is mathematically impossible without prior constraints. This is due to the fact that the image degradation process from high-resolution to low-resolution often results in multiple high-resolution images and the same low-resolution image can be obtained through degradation. Consequently, researchers in this field have turned to learning-based methods in recent years, leveraging large datasets to derive image prior information and thereby constrain the solution space in order to arrive at the optimal solution to the problem. There are three primary categories of traditional image super-resolution reconstruction algorithms:

1. **Interpolation-based image super-resolution**, which includes methods such as nearest neighbor interpolation, bilinear interpolation, and cubic interpolation. The approach calculates the value of a given point based on a formula that incorporates the values of surrounding known points, as well as the positional relationship between those points and the target point, in order to enhance image resolution. However, the algorithm's ability to improve image detail is limited, rendering it less commonly used.
2. **Reconstruction-based image super-resolution**, which involves leveraging a priori knowledge to combine multi-frame images. Starting from an image degradation model that assumes the low-resolution image is obtained from the high-resolution image via motion transformation, blur, and noise, this approach uses multiple frames to reconstruct high-resolution images.
3. **Learning-based image super-resolution**, which relies on pre-training data to establish the mapping relationship between low-resolution and high-resolution

images. By learning this mapping relationship, the approach can predict the high-resolution image corresponding to a given low-resolution image, thus facilitating the image super-resolution reconstruction process. This category has emerged as the most popular and successful approach in recent years, and is the focus of our in-depth study.



Figure 2.1: Single Image Learning Based Super-Resolution

2.2 Learning based image super-resolution

2.2.1 CNNs

Image super-resolution was first proposed by Harris[8] and Goodman[9] in the 1960s. Tsai[10] used multiple low-resolution images to restore high-resolution images in 1989. With the research and development of machine learning technology, Freeman et al.[11] applied the method of machine learning to the field of image super-resolution for the first time in 2000.

As soon as the popularisation and accessibility of deep learning started, CNN[1]-based image super-resolution algorithms started appearing. The first such architecture was SRCNN[12]. SRCNN is a fully-convolutional neural network[13], meaning it is a CNN without any dense connections (fully connected layer) at the rear of it. In SRCNN, first the image is upsampled using bicubic interpolation and then fed to a simple FCNN. It is important to note that there are no pooling operations involved. Thus, resulting in an output of the same spatial size as that of the upsampled input image. Finally, we compute the MSE loss between the target HR image with the output. This strategy was revolutionary in a sense. It provided much better results than previous approaches and started an explosion of image super-resolution algorithms based on deep neural networks.

2.2.2 ResNets

An improvement over SRCNN came in 2017 with the appearance of SRResNet[14], based on the ResNet architecture[15]. It is universally accepted that ResNets (CNN with skip connections) provide a significant improvement over conventional CNNs. Because of this, SRResNets replaced the simple convolution blocks with residual blocks. As a result accuracy increased significantly. On top of residual connections, SRResNet came with another big improvement that is used even today in super resolution architectures: Pixel Shuffle[16]. Many deep learning models incorporated traditional upsampling strategies such as bilinear or bicubic upsampling in order to increase the resolution of an image. The problem is that bilinear and bicubic upsampling are not learnable and cannot adapt. Another disadvantage is that these upsampling strategies provide no gradient information that can be back-propagated through. "Pixel Shuffle" rearranges the elements of $H \times W \times C \cdot r^2$ tensor to form $rH \times rW \times C$ tensor. With this addition in place, it was no longer necessary to use bicubic interpolation at the beginning or at the end of the architecture to increase the size of the image. Instead, most of the computation is done in the given low resolution shape, and then the resolution is only increased towards the end of the architecture using one (or multiple) pixel shuffle layer(s). The benefit of this is improved performance and especially faster computation, since the image throughout most of the architecture is smaller, as compared to previous approaches in which the image was upsampled at the beginning of the architecture and then the network worked with this larger image.

2.2.3 Perceptual loss

Another improvement in the field of image super-resolution came in 2016 when the Perceptual Loss[17] paper was released. Up until this point, the loss function used to train the super-resolution algorithms was either **MSE** (mean squared error) or **MAE** (mean absolute error). The main disadvantage of using MSE or MAE type error methods as loss function in applications like super resolution is that it is computed pixel-wise. That is, it only measures the change between two corresponding pixels in the predicted and the target images. This encourages finding pixel-wise averages of plausible solutions which are typically overly-smooth and thus have poor perceptual quality. Perceptual loss is computed by comparing two images based on high-level representations from a pre-trained CNN model. The function is used to compare high level differences, like content and style discrepancies, between images. In other words, both target and predicted input are passed through a pre-trained network and compute the euclidean distance between the two resulting feature maps (at the same stage). The perceptual loss function works by summing

all the squared errors between all the pixels and taking the mean. This is in contrast to a per-pixel loss function which sums all the absolute errors between pixels.

2.2.4 GANs

The adoption of GANs[2] (Generative Adversarial Networks) also proved beneficial in the field of image super-resolution. The GAN procedure encourages the reconstructions to move towards regions of the search space with high probability of containing photo-realistic images and thus closer to the natural image manifold. One popular image super-resolution architecture based on GANs is SRGAN[14], a GAN based network where the generator (**G**) learns to generate SR images from LR images as close as possible to HR. The discriminator (**D**) learns to distinguish generated SR images from real images. The generator (**G**) takes advantage of ResNet and sub-pixel convolution for upsampling.

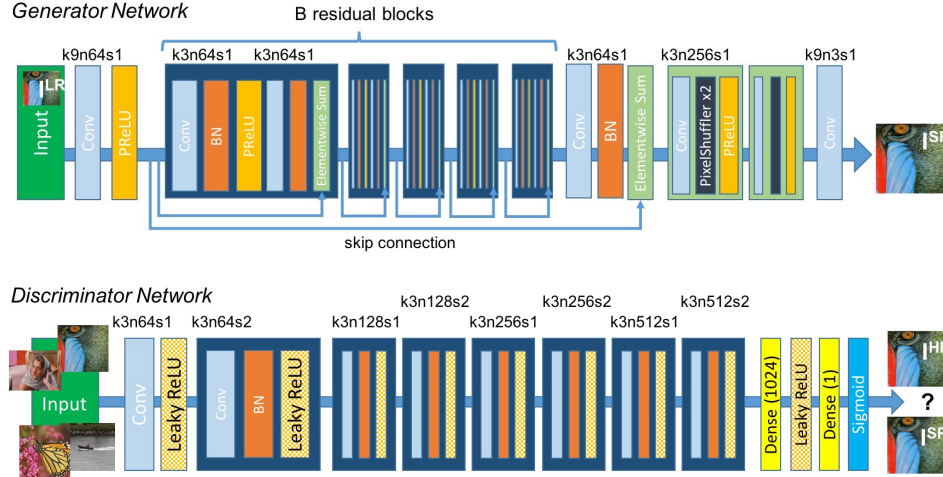


Figure 2.2: Architecture of Generator and Discriminator Network with corresponding kernel size (k), number of feature maps (n) and stride (s) indicated for each convolutional layer. Taken from the official paper[14].

SRGAN also combines previously presented perceptual loss with generative or adversarial loss for the computation of its loss.

$$l^{SR} = \underbrace{l_X^{SR}}_{\text{content loss}} + \underbrace{10^{-3}l_{Gen}^{SR}}_{\text{adversarial loss}} \quad (2.1)$$

perceptual loss (for VGG based content losses)

It is worth noting that the previously presented approach SRResNet actually comes from the same paper as SRGAN. It can be viewed as an SRGAN variant but without the Generative Adversarial Networks. Another way to look at it is that the SRGAN is based on SRResNet, to which it adds Generative Adversarial Networks.

Another GAN based method is ESRGAN[18], which is built on top of SRGAN and usually has better performance. The main contributions of the ESRGAN architecture were: **(1)** the use of Dense Blocks[19] without Batch Normalization[20] instead of Residual Blocks with Batch Normalization, **(2)** using a discriminator with relative realness instead of the absolute value, an idea borrowed from relativistic GAN[21], and **(3)** using the perceptual loss with the features before applying the activation layer instead of after.

2.2.5 Transformers

In recent years we have seen how Transformers[5] took the field of NLP by storm. The vast majority of NLP sub-domains have moved on from RNNs[22] and their variants (such as GRUs[23] or LSTMs[24]) and have now fully adopted Transformers. Seeing the great success of the Transformer architecture in NLP, the field of Computer Vision has adopted this architecture in the form of the Vision Transformer[25]. All of this is relevant since the most recent and most performant image super-resolution algorithms are based on Transformers. More specifically, they are based on the Swin Transformer[26] architecture (or Swin Transformer V2[27]). The Swin Transformer is a modified Vision Transformer with 2 big changes: the use of a modified attention mechanism called Window Attention to compute local attention between tokens within one window, and the use of the shifted windows technique which helps in computing attention between neighboring windows (see Figure 2.5).

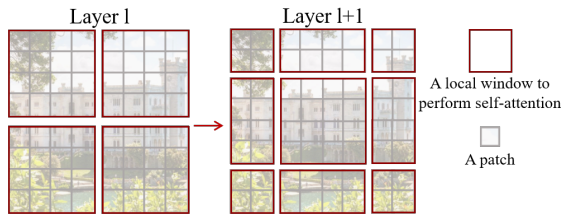


Figure 2.3: Shifted window technique used in Swin Transformer

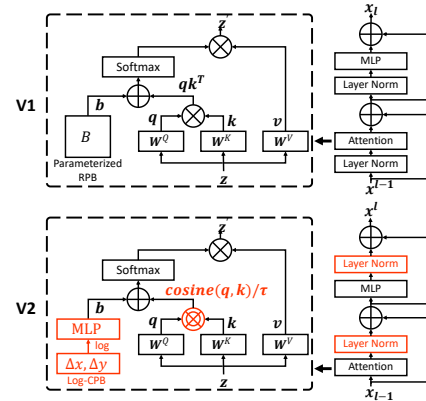


Figure 2.4: The architectural differences between SwinV1 and SwinV2

Figure 2.5: Main components of SwinV1 and SwinV2. Taken from SwinV1[26], SwinV2[27] respectively.

The main parts of this new Swin Transformer architecture were used as the building blocks of SwinIR[6] and Swin2SR[7] architectures, which are currently among the best methods of applying single image super-resolution.

Chapter 3

Optical Character Recognition

3.1 Introduction

Optical Character Recognition has gained widespread adoption across industries as a fast way to extract text data from images. However, while contour detection, image classification, and connected component analysis can effectively handle documents with comparable text size and font, good lighting conditions, and high image quality, these methods struggle to deal with irregular, heterogeneous text, also known as "wild text" or "scene text". Examples of such text can include license plates, house numbers, poorly scanned documents with no predetermined conditions, and more. To address these challenges, deep learning solutions are used.

There are 3 main components of an OCR system:

1. **Preprocessing:** Extracting text data from digital images or documents is a straightforward task. However, scanned or phone-captured images often face challenges such as noise, blur, skewness, and other issues. Image preprocessing is thus necessary to handle these challenges, including techniques such as rescaling, binarization, rotation, dilation, and erosion.
2. **Text Detection/Localization:** In this stage, models such as Mask-RCNN[28], YoloV5[29], East Text Detector[30], etc. are utilized to locate the text in the images. These models create bounding boxes (square/rectangle boxes) over each text identified in the image or document.
3. **Text Recognition:** Once the text location is identified, each bounding box is sent to a text recognition model that typically includes a combination of RNNs[22], CNNs[1], and Attention networks[5]. The output from these models is the extracted text from the documents. Popular open-source text recognition models include Tesseract[31], MMOCR[32], etc.

3.2 The link between OCR and SISR

One problem which arises fairly frequently with OCR, as is the case with most applications of deep learning, is that real world data is often noisy and of poor quality. Let's take the example of detecting the text from street advertisements. Even if we look at just this small example we can find a large number of reasons for which our data might be noisy or just not fit for our OCR model. These include: inconsistent lighting conditions, blurred or compressed images, motion blur from the camera, uneven camera focus, shaky camera, unexpected camera angles, and many more. More specifically, we want to focus on the issue of low resolution images, which happens fairly often in real world data and which has a significant impact on the performance of a subsequent OCR system.



(a) High Resolution Traffic Sign



(b) Low Resolution Traffic Sign

Figure 3.1: Visual difference between high and low resolution traffic signs. An OCR system would have considerable difficulties in detecting the text from the right image

This problem arises so often with real world data that it deserves closer inspection regarding possible solutions. To this effect, we propose an extra pre-processing step before feeding an image to an OCR system: we use a Super-Resolution algorithm on the low resolution real world image. We hypothesize that reducing the noise and increasing the resolution of an input image will offer a better starting point for an OCR system to make detections. To this end, we study the process of **Single Image Super-Resolution** (SISR). To have our approach as general as possible, we treat the OCR system as a black box without making any assumptions about it. Thus, our study will be limited to Super-Resolution techniques, and the OCR task is seen as an application of adequate image super-resolution algorithms.

Previous work has been done in this direction[33, 34, 35, 36, 37]. However, these studies used less than optimal methods of achieving their results. In these articles the authors used old/weak architectures (such as SRGAN[14]), sub-optimal degradation methods (such as bicubic downsampling), pre-trained models which were actually trained on out-of-distribution data (e.g., models trained on cartoons

super-resolution, subsequently used for textual super-resolution), or they trained their own models from scratch. In spite of these shortcomings the authors found that a super-resolution pre-processing step improved their OCR results, in some cases even by large margins. We hypothesize that these results can be improved even further if we adopt the state of the art approaches from all points of view such as the latest pre-trained models on vast datasets with state of the art degradation methods. A successful study of the proposed method could have important real-world impact by improving OCR performance, which could have applications in areas such as document processing, data entry, and information retrieval.

Chapter 4

SISR - Discussion

4.1 Datasets

There are two possible approaches in developing a dataset for an Image Super-Resolution task: **(1)** take two images of the same exact scene with different camera parameters, resulting in a low resolution image and a high resolution image and **(2)** have a set of high resolution images and apply degradation methods on them in order to obtain their low-resolution counterparts. Approach **(1)** is very expensive and time consuming, so the vast majority of researchers have used and will continue to use approach **(2)**, in which we synthesise our own dataset starting from images of high resolution.

For training a Super-Resolution system starting from high resolution images there are a number of datasets which have been used over the years. These include: random samples taken from ImageNet[38], DIV2K[39], Flickr2K[40], OST[41], WED[42], FFHQ[43], Manga109[44], SCUT-CTW1500[45], and BSD500[46]. For testing and reporting metrics for a Super-Resolution method the datasets most used are: Set5[47], Set14[48], BSD100[46], PRIM[49], Urban100[50], Manga109[44], CBSB68[46], Kodak24[51], McMaster[52], Classic5[53], and LIVE1[54]. Also, some of the datasets used for benchmarking and metric reporting used the more expensive method **(1)** of acquiring a dataset with paired HR-LR images, built with carefully designed techniques and advanced digital devices. Representatives are: City100[55], DRealSR[56], and RealSR[57].

4.2 Degradation methods

As already discussed, since acquiring pairs of low-resolution and high-resolution images is expensive and time consuming, researches mostly use degradation methods on high resolution images to obtain low-resolution counterparts and thus to

synthesise their dataset. Since this is the case, it is important to have degradation methods which produce images that look as similar as possible to the type of low resolution images we would meet in real life.

For the majority of the time, this has not been the case. Many existing methods for image super-resolution assume an ideal bicubic downsampling kernel, which does not accurately reflect real-life image degradation. Furthermore, due to historical factors, researchers and super-resolution competitions have typically relied on the MATLAB bicubic downsampling method. As a result, attempting to reproduce the results of a published paper using a different bicubic downsampling technique, such as the one found in OpenCV or PyTorch, may lead to failure. Fortunately, this issue has started to be addressed more frequently in recent years.

If we move away from the less than satisfactory method of bicubic downsampling we can mathematically model the degradation with the following equation:

$$\mathbf{x} = \mathcal{D}(\mathbf{y}) = [(\mathbf{y} \otimes \mathbf{k}) \downarrow_r + \mathbf{n}]_{\text{JPEG}}, \quad (4.1)$$

where $\mathbf{x}$ is the low-resolution synthesized image, $\mathbf{y}$ is the ground-truth high-resolution image, $\mathbf{k}$ is a blur kernel, $\downarrow_r$ is a downsampling operation with scale factor r , $\mathbf{n}$ is white Gaussian noise, and $[]_{\text{JPEG}}$ is a JPEG compression operation.

The paper BSRGAN[58] was the first one to adopt a hand-designed degradation model for general blind image super-resolution (here “blind” refers to the fact that no camera parameters are known). The main contribution of the paper was to construct and set the parameters of the sequence of transformations needed to create a low-resolution image from a high-resolution one. The transformations they have agreed upon are: (1) *blur* for both the HR image and the LR image using *two Gaussian blur operations*, i.e., $\mathbf{B}_{\text{iso}}$ with isotropic Gaussian kernels and $\mathbf{B}_{\text{aniso}}$ with anisotropic Gaussian kernels[59, 60, 61], (2) random *downsampling* strategy chosen between nearest neighbor interpolation, bilinear, and bicubic, (3) *gaussian noise*, either white Gaussian noise (AWGN) or gray-scale AWGN noise, (4) *JPEG compression noise* with randomly chosen compression degree, and (5) *processed camera sensor noise* through a reverse image signal processing (ISP) pipeline followed by the forward pipeline after adding noise to the synthetic raw image in order to reconstruct the noisy RGB image. These transformations are then randomly shuffled before being applied in sequence on an image.

A similar thing happened in the paper Real-ESRGAN[62], in which the authors put significant effort into increasing the chance that the degradation methods are as close as possible to real life image degradation. The significance of this paper is that they studied the effects of a *high-order degradation model*. In essence, applying degradation in a high-order manner means applying the degradation strategies multiple times and not just once. This can be modelled mathematically using the

following equation:

$$\mathbf{x} = \mathcal{D}^n(\mathbf{y}) = (\mathcal{D}_n \circ \dots \circ \mathcal{D}_2 \circ \mathcal{D}_1)(\mathbf{y}). \quad (4.2)$$

The rationale for this approach is that real-world degradation processes are highly varied, often involving a complex sequence of steps that may include camera imaging, image editing, Internet transmission, and so on. To illustrate this point, the authors provide an example of a low-quality image downloaded from the Internet. The degradation processes underlying this image are multifaceted, involving a range of different factors. For instance, the original image may have been taken using a cellphone several years ago, leading to issues such as camera blur, sensor noise, low resolution, and JPEG compression. The image may have then been edited using sharpening and resizing operations, which can introduce unwanted artifacts like overshoot and blur. Finally, the image may have been uploaded to various social media platforms, introducing further compression and noise. As the image spreads further on the Internet, additional artifacts may be introduced, making the overall degradation process even more complex. Such complicated cases cannot be modelled by a simple first-order degradation model. Empirically, the authors found that a second-order degradation model can resolve most real cases while keeping simplicity.

The authors of both papers found their respective degradation methods to be highly effective in bridging the gap towards real-life image super-resolution. The models on one hand achieve better metrics on the standard super-resolution benchmarks and on the other hand are visually more pleasing.

4.3 SISR metrics and their shortcomings

There is still an ongoing discussion regarding the method of evaluating a super-resolution model. It is universally accepted that **MSE** (mean-squared error) or **MAE** (mean-absolute error) between the ground-truth and the super-resolved image are not satisfactory evaluation metrics since they do not take into account factors such as contrast, structural similarity, and perceived quality, which are important for evaluating the visual quality of an image. The most widely used metrics are **(1) PSNR** (peak signal-to-noise ratio):

$$\text{PSNR}(x, y) = 10 \log_{10} \left(\frac{L^2}{\text{MSE}(x, y)} \right) \quad (4.3)$$

where L^2 is the range of the pixel values and $\text{MSE}(x, y)$ is the mean squared value of the differences between the two images x and y ; and **(2) SSIM** (structural similarity index):

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (4.4)$$

where μ_x is the pixel sample mean of x , μ_y is the pixel sample mean of y , σ_x^2 is the variance of x , σ_y^2 is the variance of y , σ_{xy} is the covariance between x and y , and c_1 and c_2 are two variables which stabilize the division with weak denominator.

Another way of evaluating the performance of a super-resolution method is the one used in the SRGAN[14] paper, where the authors used the **MOS** (mean opinion score) testing meaning 26 human raters had to assign scores to pairs of images between the ground-truth and the super-resolved images. This, however, is an expensive procedure and the majority of authors do not rely on it.

The main problem which arise from using metrics such as **PSNR** or **SSIM** is that a higher value for them does not always correspond to a visually more pleasing super-resolved image. Often times, a method which yields lower values for these metrics will outputs better images. One can see such examples in Figure 6.1.

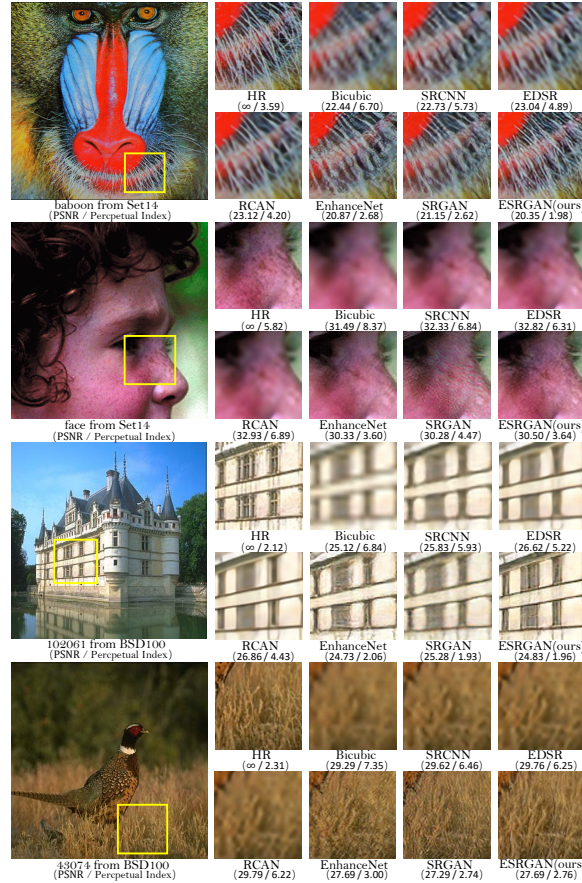


Figure 4.1: Examples of super-resolved images with visually more pleasing outputs, but with lower PSNR scores. Taken from ESRGAN.[18]

Chapter 5

Experiment - Proof of concept

5.1 POC idea

Before carrying on with our application we want to verify if our efforts will be worthwhile using a small experiment. We want to see if our hypothesis is true for at least a small dataset before moving on to developing our application. For this experiment we will use the Super-Resolution dataset from ICDAR-2015 competition on Text Image Super-Resolution[63]. It consists of a total of 708 images designed for OCR evaluation. Every image has two versions of itself: the original high definition image and the low resolution version of the image, which was created through degradation methods by the authors of the dataset. Our goal with this small experiment is to verify the performance of an OCR system on the original high resolution images and use that as a baseline. Then, we will evaluate the same OCR dataset on the low resolution version of the images. We expect this result to be less satisfactory than the result we got using the high definition images, irrespective of the OCR system or the metric we use. Finally, we want to use a super-resolution method to upscale our low-resolution images and verify the performance on this final version of the dataset. If our hypothesis is true, then we expect the super-resolution images to have higher performance than the lower resolution images, and our hope is that the performance will be close to the performance achieved by the original, high-resolution images.

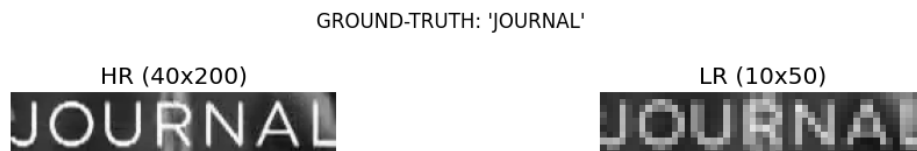


Figure 5.1: Image pairs from ICDAR-2015 Text Image Super-Resolution competition[63]

5.2 Attempts to simple OCR performance boosts

In order to find if the super-resolution pre-processing step is actually helpful and exactly how helpful it is in comparison to other methods, we will compare it not only to the results we get on the low-resolution and high-resolution versions of the images, but also to other simpler attempts of improving OCR performance. More specifically, we are interested in finding the OCR performance boost if we use simple upsampling techniques (e.g., bicubic upsampling) or simple color manipulation techniques (e.g., increasing image contrast). The performance gain which we will have through an image super-resolution pre-processing step (if any) must be significantly higher than what we can achieve with simple approaches like bicubic resizing for it to justify the added complexity and computational requirements which come into the picture when we add a deep learning model in the pipeline.

5.3 Architecture choice

With the increased interest in Single Image Super-Resolution we observed a large number of models and architectures which were used over the years, as already presented in previous chapters. With this large body of architectures and general methods of applying deep learning to image super-resolution comes the question of which model we should use. To this effect, we will study the performance of multiple models on our toy dataset, namely we will evaluate the performance of the following models: ESRGAN[18], Real-ESRGAN[62], BRGAN[58], SwinIR[6], and Swin2SR[7]. Each of these models have been used extensively by the both researchers and the SISR community and are known to perform well in this domain.

In order to evaluate these models we will look at three points of view: two performance related points of view and one point of view which will represent the computational requirements of the model. More specifically, for each model, we will look at: PSNR and SSIM (performance related metrics) and model size (computational requirements). For the latter, it is a matter of simply counting the number of parameters a model has. For the former, we will apply super-resolution with each model we have on the low-resolution version of the images and find the metrics using the provided high-resolution images as ground-truth.

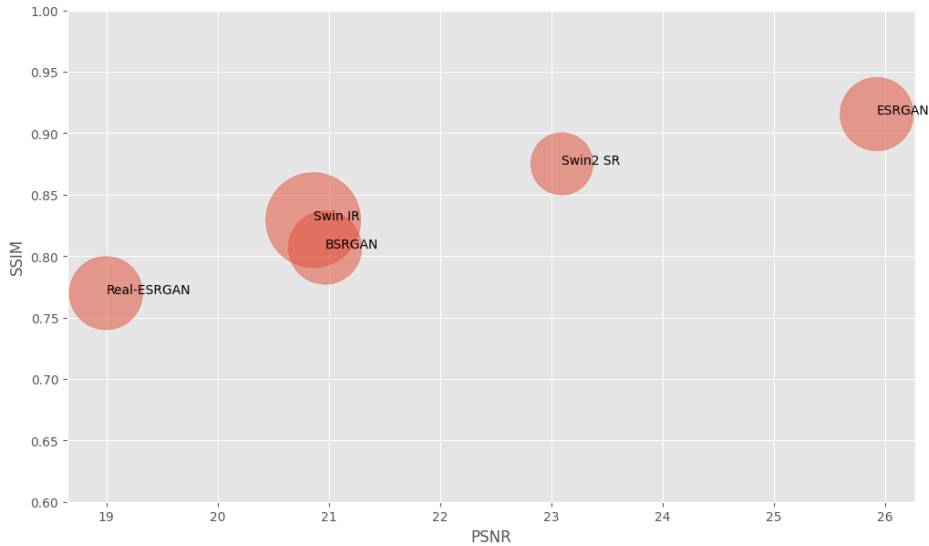


Figure 5.2: Models comparison - performance and size

In the above figure we compare the models using their PSNR score as the x-axis, their SSIM score as the y-axis, and the model size is proportional to the size of each bubble. Using this information, we can deduce that the best model to use for our particular dataset (ICDAR-2015) is the ESRGAN, with a few important caveats:

1. ESRGAN is a relatively old model, trained with poor degradation methods and which usually achieves poorer results than any of the other models.
2. The ICDAR-2015 dataset has very few images (708), so making a conclusion based on the metrics from this small dataset might give us a narrow view of what is actually going on.
3. The scales in the x-axis and y-axis of the above plot are worth noting: If we take those into consideration we can observe how in actuality all the models achieve similar and comparable performance.
4. The two performance metrics we used are notoriously non-reliable. While PSNR and SSIM are indeed better than using a simple MSE or MAE, they are still far from perfect and it was observed that oftentimes a higher PSNR or SSIM metric actually results in a less visually appealing image [18].

To extend the last point from the above list, we can mention that on top of the fact that PSNR and SSIM are not reliable, we can also make the case that we shouldn't aim to perfectly replicate a high-resolution image from its low-resolution counterpart. Since we are interested in boosting OCR performance first and foremost and

not in improving SISR metrics, we can make the case that a super-resolution image is BETTER suited for an OCR task than its high-resolution counter-part even if it is structurally different than the high-resolution image. In short, we are interested in making an image clear for a subsequent OCR task, not in perfectly recreating a high resolution image. To make this matter more concrete, we will look at a few examples:

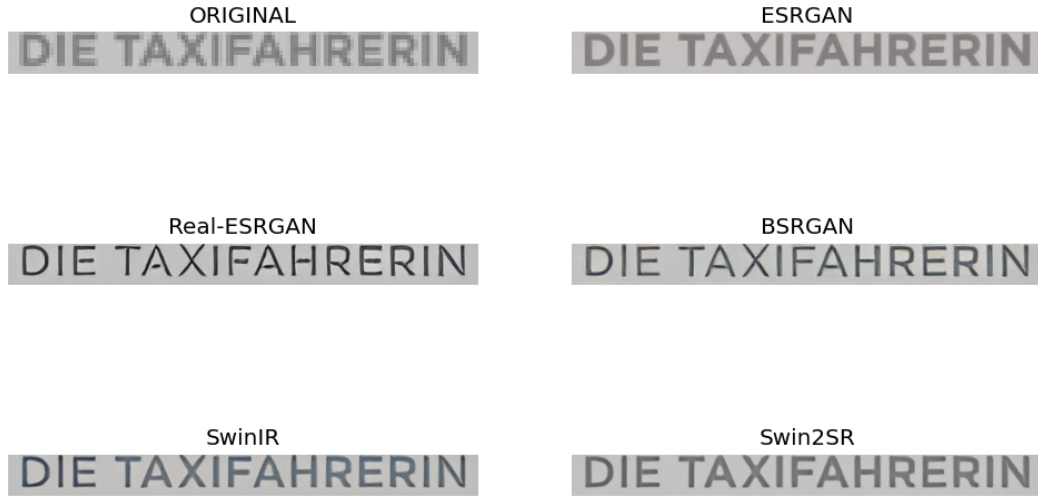


Figure 5.3: HR vs LR vs SR versions for some images

In the above example we can see the issue which comes with exclusively analyzing super-resolution metrics. In these examples, the super-resolution reconstruction is quite a bit off from the original high-resolution images, so the PSNR and SSIM metrics would not be anywhere near perfect. However, since we are mainly interested in improving OCR performance, we could make the case that the super-resolution reconstructions are actually better than not only the low resolution images, but even better than the original high-resolution versions. The super-resolution versions are not only higher in resolution than the low-resolution images (and equal in resolution with the high-resolution) counter-parts, but also clearer and sharper than even the original, ground-truth images. This is a side effect of using modern super-resolution models, which also have a slight denoising effect. With this idea in mind, we must be careful about metrics-based decisions and always take the PSNR and SSIM scores into consideration with a healthy dose of skepticism.

Given that we cannot fully rely on the PSNR and SSIM metrics, we have to visually check our images and/or use different models. We can see in Figure 5.4 the visual differences produced by our models on the dataset. As a general trend, we can see that ESRGAN is usually less aggressive than more modern SISR methods. This can be beneficial in some cases, but in other it simply doesn't provide enough sharpness. On the other hand, Real-ESRGAN tends to be overly sharp and some-

times creates unpleasant artifacts. The swin variants are usually the most appealing on this particular dataset. Most likely this is because of the much larger datasets which were used to train the swin variants, including a large number of images with text on them.



(a) ESRGAN is in this case less aggressive resulting in a more realistic image



(b) The swin variants produce sharper images

Figure 5.4: Visual comparison of our models

It is also worth to mention the differences in training datasets which were used for these models, since that can have a large impact on the performance of our subsequent task. What we are looking for is a model which has been pre-trained on text data, among other types of data. Since we are mainly interested in making text clearer, we want a model which has been trained for this. This is especially important for this proof-of-concept experiment since we will not conduct any fine-tuning

because of the small sample size, we will use the pre-trained models as they are. Of the above five models we can note:

1. ESRGAN - no explicit text data has been used in training
2. Real-ESRGAN - no explicit text data has been used in training
3. BSRGAN - no explicit text data has been used in training
4. SwinIR - specifically included SCUT-CTW1500[45] in its training data for the text
5. Swin2SR - specifically included SCUT-CTW1500[45] in its training data for the text

It is also worth noting that the super-resolution models which did not have explicit text data in their training datasets still perform well for our OCR task. This is potentially because text is very often present in everyday images such as street signs, advertisements, brand names, etc.

5.4 OCR improvements achieved through SISR

To evaluate the performance of an OCR system for one image, we will look at the string similarity between the ground-truth text from the image and the predicted OCR text. The exact formula for calculating this similarity is based on the Levenshtein distance between the two strings, and is implemented in our solution through the **FuzzyWuzzy** library in Python (more specifically, we use the **ratio** method provided by the library, which calculates a normalized Levenshtein distance between two strings). Using this per-image similarity measure, we will find two numbers which will represent a model's performance as a whole: the mean and median similarity measure across all the images in our dataset. We use the mean to get a general idea of the overall performance and we use the median to get a more robust view of how our model is behaving, since the OCR system often fails for images which it intuitively shouldn't, so we get a lot of images with 0 similarity measure which will drive down the mean. As for the OCR system, there are multiple open-source OCR systems we can use. For this POC experiment we use Google's Tesseract OCR [31].

As we can see in Figure 5.5, there are multiple cases where the super-resolution algorithm brings improvements to our OCR task. The SR model seems to find enough information in the low-resolution image to reconstruct a high-resolution image which properly depicts the original text. What is more, one could make the argument that sometimes the SR model provides images which are even better than



Figure 5.5: Where SR improves OCR performance

the original high-resolution ground truth images, giving sharper images thanks to the denoising effect of these super-resolution algorithms.

However, it is worth noting that the SR pre-processing step has many cases where it fails to bring improvements. When the low-resolution image doesn't have enough information for a proper high-resolution reconstruction (i.e., the low-resolution image simply has too few pixels for the SR model to detect the characters which should be reconstructed), the model creates unpleasant artifacts, as we can see in Figure 5.6.

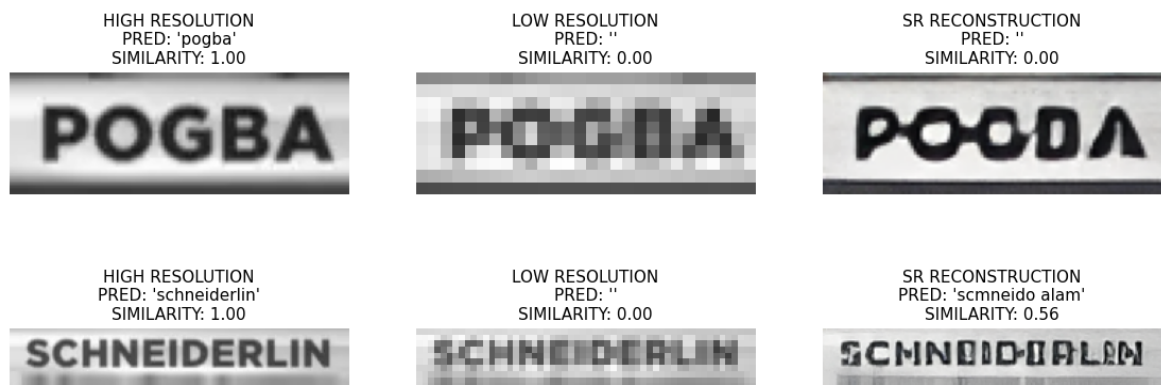


Figure 5.6: Where SR doesn't improve OCR performance

In spite of these rare failures such as the ones from above, the SR pre-processing step as a whole seems to be overall beneficial when we look at OCR performance. We can see in Figure 5.7. Using SR as a pre-processing step boost OCR performance significantly in both our our metrics (mean and median similarity). The performance is better than what we can get by simply using a bilinear upsampling and is very close to the performance we get on the original high-resolution images (same mean and just 1% off in terms of median when compared to the OCR performance achieved on the original high-resolution images). Also, it is worth noting that artificially raising an image’s contrast did not seem to help, since combining this step with the super-resolution method actually yields worse results than using just the SR method.

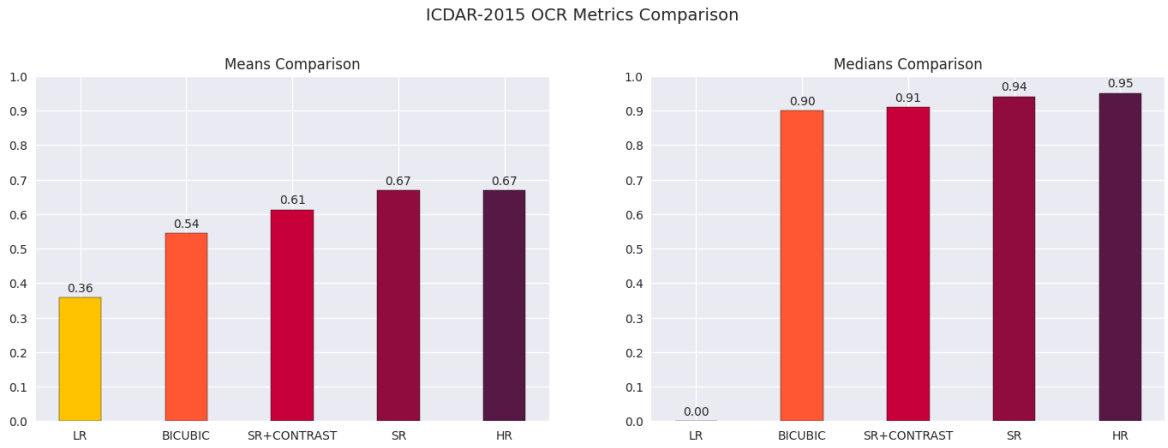


Figure 5.7: SR brings significant improvements to OCR performance on ICDAR-2015

In this study, we explored the potential benefits of using an image super-resolution pre-processing step to improve the performance of an OCR system. We conducted a POC experiment using the ICDAR-2015 OCR dataset and found that the use of super-resolution does indeed improve OCR performance, with an average accuracy increase of 37% in the mean similarity score. These findings suggest that further investigation into the use of super-resolution in OCR tasks could be valuable. However, it is important to acknowledge that this study has some limitations, such as the relatively small size of the dataset used. Further research is therefore needed to confirm these findings and explore the potential benefits of super-resolution in larger datasets and different contexts.

Chapter 6

Further Investigations

6.1 The need for additional experiments

We have seen in our proof-of-concept experiment from Chapter 5 that a super-resolution pre-processing step can improve the performance of an OCR system. This is a good step forward. However, further investigations are needed. The dataset used in our experiment, ICDAR-2015[63], is the best suited dataset we could've used for this task. First of all, the images provided by this dataset are already cropped and centered around text. The subsequent OCR system, whichever one we choose, does not have to perform any sort of localization, just text detection. Secondly, the images are not naturally obtained low-resolution images. Instead they are degraded high resolution images. We do not know the exact degradation methods used by this dataset but since it is a relatively old dataset we can assume it used the very frequent (at the time) bicubic down-sampling. Thirdly, the text in the provided images is all with a similar font and the text is very nicely placed horizontally. We do not have images with unusual font or curved vertical texts in ICDAR-2015.

In short, before we jump to the conclusion that an OCR pre-processing step is definitely beneficial to a subsequent OCR task we must carry out further investigations on more diverse datasets and analyze the results there. To get this more complete image of how our OCR task responds to image super-resolution we will conduct experiments on the following three datasets:

- **FUNSD:** dataset consisting of scanned documents
- **Total Text:** extension of ICDAR-2015. It consists of images with naturally occurring text (e.g., street signs, advertisements, etc)
- **Low Resolution Scanned Text Dataset:** benchmark text recognition dataset low-resolution book texts.
- **Incidental Scene Text 2015 Dataset:** real scene images which appear in the scene without the user having taken any specific prior action to cause their

appearance or improve their positioning/quality in the frame.

These three datasets provide a wide and varied source of images which will help us see the influence of the SISR pre-processing in a clearer view. It is important to highlight the fact that we explicitly excluded the SCUT-CTW1500[45] dataset since that dataset is part of the training dataset used in SwinIR[6] and Swin2SR[7].

6.2 The FUNSD dataset

The FUNSD(Form Understanding in Noisy Scanned Documents) dataset[64], consists of 199 images of fully annotated documents. The dataset can be used for Text Detection, Optical Character Recognition, Spatial Layout Analysis and Form Understanding, having a total of 31485 words, 9707 semantic entities, and 5304 relations. For our purposes, we will use this dataset just for the OCR task since that is all that we are interested in. An important note is that the documents are actually scanned forms so some of them will have not only originally printed text but also some sort of added text, either also printed or hand-written.

**Allowable Grade Substitutions
for Export CPCL**

Standard Grade Mark	Allowable Substitutions
PCFS Flue cured fines	RPCFS or RPCFS-O
PCBS Burley fines	RPCBS or RPCBS-O WCC
MC-4 Manufacturing fines	MC-4-S MC-4-PH MC-4-RC MC-4-W
MC-6 Stem Meal	MC-1
MC-7 Winnowers	C Stem
RXF Fine flue cured stem	RRXF or RRXF-O RRXM-B RRXM-T
RBF Fine Burley stem	RRBF or RRBF-O RRB-B RRB-L RRB-T

421500455

Figure 6.1: Example of scanned document form from FUNSD[64]

6.3 Tile SISR inference

One important aspect which we must mention is **tiling super-resolution inference**. The main problem we run across when running inference with a super-resolution model (it would be even worse with training instead of inference) is that these models consume a lot of memory. Even though recent architectures have put considerable effort into reducing the number of parameters of a SISR model, their memory requirements are still very large since they inherently work with image input and image output. What is more, unlike an architecture such as the Autoencoder[65] or U-Net[66] which reduce the spatial representation of the image through the network and then recreate it in the second half of the architecture, the image super-resolution architectures work with feature representations with the same height and width as the original image and with significantly increased channels dimension. For this reason the memory requirements are quite significant, especially for images which already have a high resolution. If we take into consideration the model's size (which is by itself quite large), the memory consumption of each layer by multiplying $B \times C \times H \times W$ with the data type size (usually `float32`), and also consider additional memory overhead such as intermediate tensors or gradients we find that even with very advanced GPUs we still get the infamous **CUDA: Out of memory** error when the input image is of considerable size (high resolution).

To address this issue we will refrain from running inference on the entire image at once and instead we will break the image down into patches (we will call them "tiles"), run SISR inference on each tile, and then concatenate the tiles into one output image. With this approach the memory consumption requirements are no longer proportional to the image size (at least not after a threshold, namely after the input image size is larger in height/width than the tile size). This strategy is similar to how we apply kernels during the convolution operation in convolutional neural networks. Another similarity to the convolution is that we will also apply tile overlap in order to provide the structures toward the edges of the tile more context. Because of this and of the fact that we need the output to be interpretable as an `int8` value we will divide the pixels which accumulate output from multiple tiles by the number of tiles it accumulated output from. This will produce correct results on average. We can see an orientative implementation of this tile SISR inference in Figure 6.2. With this in place, we can apply our SISR models on images of any size. While this trick might produce slightly different results compared to applying the model on the entire image, we found that in practice the differences are not noticeable. Moreover, from a certain point this is no longer a question of what produces better results but what produces any results. For an already large image we cannot apply

a SISR model even on the latest tensor-specialized GPU's because of the memory requirements, so in some cases we absolutely have to apply this tiling technique.

```

if tile is None:
    # if no tiling => run the model on the entire image in one go
    sr_img = model(lr_img)
else:
    b, c, h, w = lr_img.shape
    tile = min(tile, h, w) # tiles should be smaller than the height/width
    assert tile is None or window_size is None or tile % window_size == 0, \
        "ERROR: Tile should be a multiple of window size."

    stride = tile - tile_overlap
    h_idx_list = list(range(0, h - tile, stride)) + [h - tile]
    w_idx_list = list(range(0, w - tile, stride)) + [w - tile]
    e = torch.zeros(b, c, h * scale, w * scale).type_as(lr_img) # where the output image will be saved
    w = torch.zeros_like(e) # count how many tiles are added on top of each other on each pixel

    for h_idx in h_idx_list:
        for w_idx in w_idx_list:
            # get a tile and run the model on it
            in_patch = lr_img[..., h_idx:h_idx + tile, w_idx:w_idx + tile]
            out_patch = model(in_patch)

            # add the tile output to the resulting image and also increase the tile count for these pixels
            out_patch_mask = torch.ones_like(out_patch)
            e[..., h_idx * scale:(h_idx + tile) * scale, w_idx * scale:(w_idx + tile) * scale].add_(out_patch)
            w[..., h_idx * scale:(h_idx + tile) * scale, w_idx * scale:(w_idx + tile) * scale].add_(out_patch_mask)

    # divide the pixels of the output image by the number of tiles which added to that pixel
    sr_img = e.div(w)

```

Figure 6.2: Tile inference implementation

6.4 How to match bounding boxes

Before we can run any experiments on the FUNSD dataset, there is another important difference between the type of the data from this dataset when compared to the data of our previous experiment, which used the ICDAR-2015 dataset[63]. This problem is that in the ICDAR-2015 dataset our data was already cropped around some text, so our OCR system essentially only needed its text recognition capabilities, and not its text detection capabilities. This is not such a big problem on the model's side since open-source OCR systems such as Google's Tesseract or MMOCR have support for both detection and recognition of text. It is, however, a problem on our side, a problem which we need to solve. The issue is that we need to be able to perform some sort of matching between the list of ground-truth bounding boxes and the list of predicted bounding boxes.

Let's say that for an image I we have a list of n ground-truth bounding boxes x_i ($0 \leq i < n$) and a list of m predicted bounding boxes y_j ($0 \leq j < m$). For any bounding box x_i or y_j we know: (1) the rectangle which defines the bounding box, (2) the text corresponding to the bounding box part of the original image, and (3) the confidence of the prediction (in the case of the predicted bounding boxes). We must have a way of matching each ground-truth bounding box to a predicted bounding

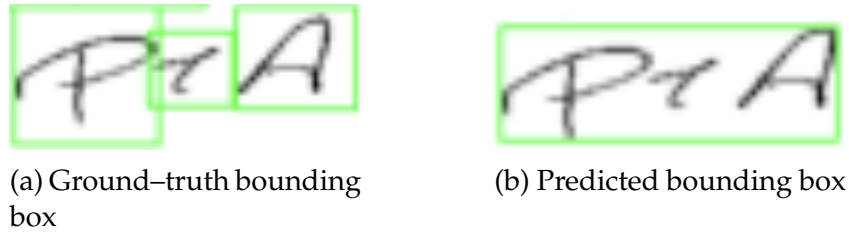


Figure 6.3: We might want the same ground-truth bounding box to be matched to multiple predicted bounding boxes.

box. One possible matching algorithm is to use a type of bipartite matching where each ground-truth bounding box is matched 1-to-1 to a predicted bounding box. But, because of the fact that n and m can be different combined with the fact that sometimes it might be desirable to have multiple ground-truth bounding boxes be matched to the same predicted bounding box (see Figure 6.3), we do not use the bipartite matching approach.

Instead, we do something much simpler. We want to have something to compare the ground truth bounding boxes and text to, so we will try to find a corresponding predicted bounding box for every ground truth bounding box. To do this, we will take every ground truth bounding box and calculate its **IoU(intersection over union)** with respect to each predicted bounding box. Then, we will match this ground truth bounding box with the predicted bounding box which achieves the highest IoU with the ground truth bounding box. This way, every ground truth bounding box will have a corresponding predicted bounding box match. If we implement this approach and look at a few results, we can see that in the vast majority of the time this method results in correct matching:

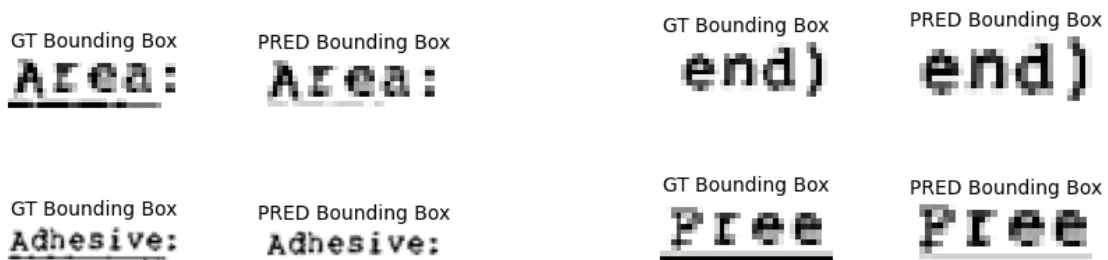


Figure 6.4: Results of our bounding box matching approach

In some cases where the IoU between the ground truth bounding box and the predicted bounding box is small even for the best match we might still get an incorrect match (see Fig 6.5). However we will try to counteract this issue as much as we can by dis-considering any pair of (ground truth bounding box, predicted bounding

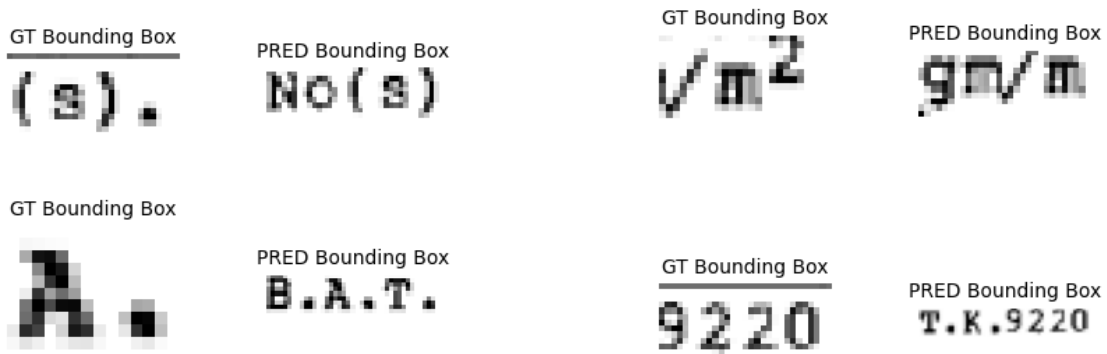


Figure 6.5: Where our matching approach fails

box) if the IoU between the two is below a certain threshold which we set experimentally. For all the ground truth bounding boxes which have a matching predicted bounding box with an IoU smaller than the set IoU threshold we will count a zero similarity and consider that our OCR system fails to detect that text.

6.5 The good, the bad, and the slow

After implementing the above tricks we can successfully carry out the analysis of applying SISR on the FUNSD dataset and then analyzing the OCR results on the super-resoluted images.

6.5.1 The good

Looking at the performance of the Tesseract OCR system on the FUNSD dataset we see that we have an **11% increase in average text similarity**. This result is significant and shows that SISR can be applied on real-world data for OCR enhancement and not only on a toy dataset such as the ICDAR-2015 dataset.

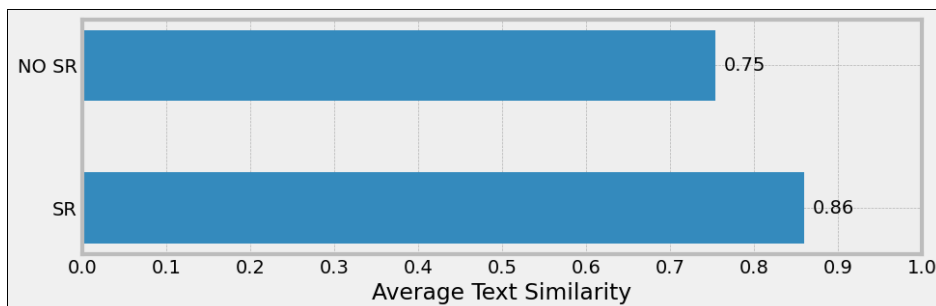


Figure 6.6: OCR on FUNSD dataset, with and without Super-Resolution

This improvement is further reinforced by visual examples. We can see exactly where the SISR model provided beneficial detail to input images and allowed the OCR system to better recognize characters.

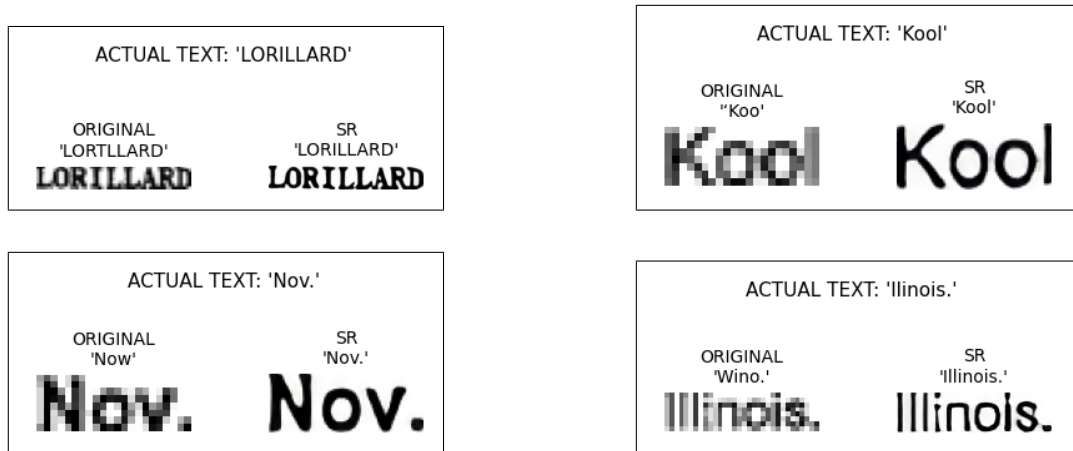


Figure 6.7: Where SR helps the OCR system

The above figures exemplify cases where the OCR **recognition** model is improved since the detection were there in the original image as well, it's just that the characters were not properly recognized. On top of these cases we can also find cases where the **detection** model of the OCR system is improved, meaning cases where a certain piece of text was not even detected before, so the similarity with the ground truth was naturally zero. But now, thanks to the increased image resolution, there are more texts which are detected than in the original image (see Fig 6.8; left - previously not detected, right - detected on the super-resoluted image).

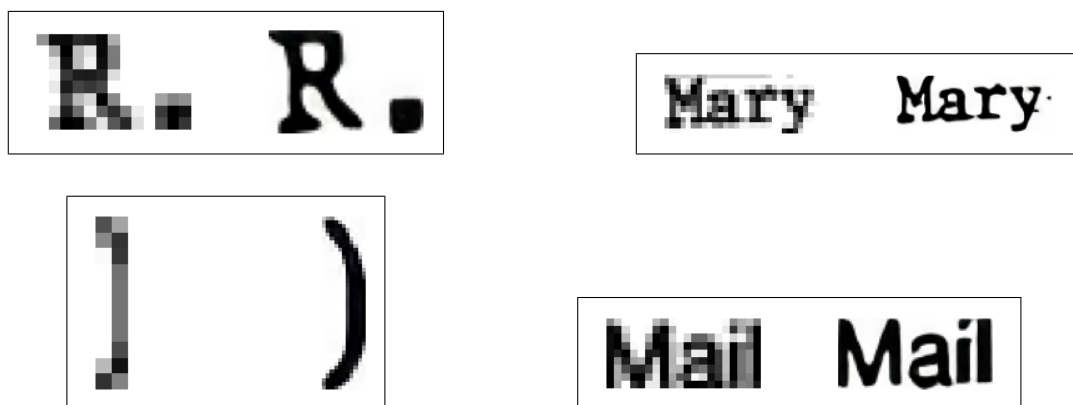


Figure 6.8: New detections given by the SISR pre-processing

Looking at both the visual and the statistical results we can conclude that SISR is also **beneficial on the FUNSD dataset**.

6.5.2 The bad

Although the SISR pre-processing step is on average beneficial to the OCR task, we must also mention the cases where applying the SISR pre-processing step **hurts performance**. The following examples show a few cases where the introduced hallucinations make certain characters less recognizable:

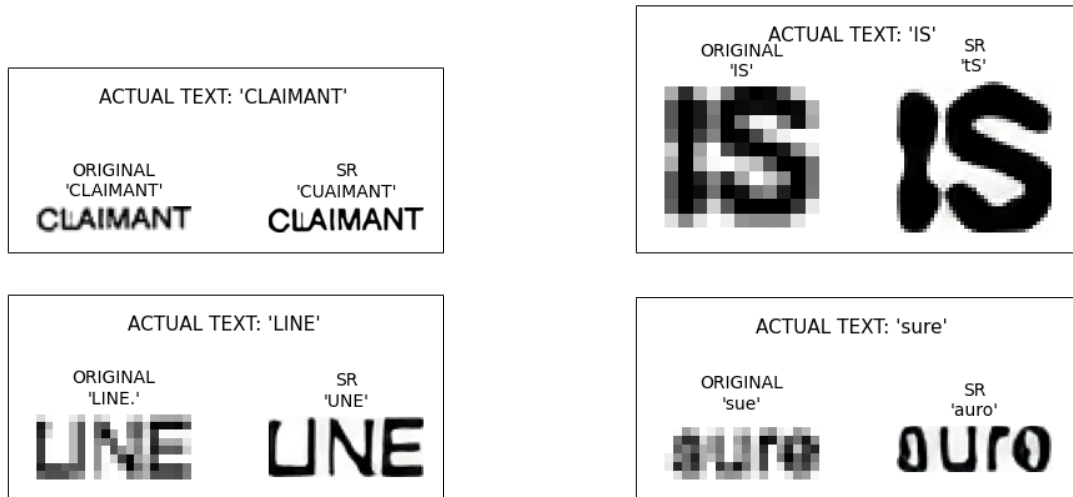


Figure 6.9: Where SR degrades OCR performance

This usually happens in the cases where the original image has a very low quality and thus the super-resolution model does not have enough information for image reconstruction. But again, although this phenomenon does happen, it happens very rarely and the large number of times when SISR is beneficial more than makes up for these few mistakes and significantly improves OCR accuracy.

6.5.3 The slow

There is one other aspect we must take into consideration when analyzing the performance gains brought by SISR: the increased processing speed. With this extra pre-processing step in place there are two steps which will naturally increase the processing speed:

1. Applying the super-resolution deep learning model on a given image
2. Running the OCR system on a now larger image

Point 1 refers to the extra processing time which will result from applying the super-resolution model on an image. This will vary depending on the size of the original image, but regardless of what super-resolution architecture we will use,

they all work with feature vectors which have a height and width equal to the input image's height and width so the computational requirements and therefore the increased processing time will be significant. Point 2 refers to the increased OCR processing time that will result from using a larger image. As already discussed, an OCR system is in essence two deep learning models: a text detection model and a text recognition model. The text detection model takes a full image as input and outputs regions where it detected text. The text recognition model takes in the regions of detected text which were outputted by the detection model and outputs the string which represents the text in the given region. If we increase an image's size in the way we do with a SISR model (e.g., we increase the resolution of our images 4x), then the text detector will naturally take longer to run. Additionally, since all of the text regions will also be bigger in resolution compared to their original counterparts, the text recognition model will also take longer. Taking all of these methods into consideration, we achieved an average processing time increase of 15x on the FUNSD dataset. This increased processing time was achieved without any batch inference in order to replicate real-time, per image inference. Batch inference would of course decrease this large processing speed, but then we would lose the element of on-demand inference.

Whether the increased processing time is acceptable or not depends on a large number of factors, including but not limited to:

- **Application Requirements:** The specific requirements of the application we are building might deem the improved performance as unsatisfactory when taking into account the extra processing time. For example, if we are developing a real-time system where immediate responses are critical (e.g., autonomous vehicles), shorter processing times would be crucial.
- **User Experience:** Even if the response time is not critical because of a real-time setting, if the method is part of a user-facing application, then the acceptable processing time would depend on the user experience expectations. Users might find longer processing times frustrating or disruptive, leading to a negative experience.
- **Computational Resources:** The available computational resources, such as processing power and memory, can influence the acceptable processing time. If the system has limited resources, shorter processing times might be necessary to ensure efficient utilization. This is especially worth considering since SISR methods have very high computational demands.

In conclusion, we have seen that using a SISR pre-processing step in an OCR task is indeed beneficial for the FUNSD dataset. For this dataset, we have achieved an 11% increase in average text similarity, from 75% to 86%. The downside of this approach is that our processing time increased drastically (15x to be exact). This large processing time increase mainly stems from the fact that images from the FUNSD dataset are already quite large (most images have a resolution around 700x1000). Whether the increased performance is worth the large processing time or not should be determined after taking into account the specifics of the application in question.

6.6 The Total Text dataset

The Total Text[67] dataset acts as an extension of the ICDAR-2015 dataset. It consists of a total of 1555 real-world images and 11459 annotated text instances. Also, it is the first dataset which features 3 different orientations: horizontal, multi-oriented, and curve. It was created after the realization that curved text is commonly seen in real world scenery, but it has close to zero existence in existing datasets and so, as a result, text detection algorithms with curved oriented text in consideration is rarely seen.



Figure 6.10: Image from the Total Text dataset

Given that the OCR system used in our task can only output horizontal bounding boxes, we will not take advantage of the multi-oriented and curved bounding boxes offered by this dataset. We already have two experiments under our belt and so far

it seems like SISR significantly helps the OCR task. First, we tested our hypothesis on the ICDAR-2015 dataset in chapter 5. The ICDAR-2015 dataset is an ideal dataset for this task since it already provides versions of low and high quality image pairs and we can easily compare the images resulted from using our SISR model with the high-quality images in terms of OCR performance. Additionally, the ICDAR-2015 dataset already offers crops around texts so the detector in our OCR system does not have to do anything, just the recognition model. There we have found that the performance reached with the super-resoluted images is almost as high as the performance of the OCR system on the original high resolution images, and significantly higher than the performance on the low resolution images. Then, in section 6.2 we explored our idea on the FUNSD dataset, which contains images of scanned documents with both printed and hand-written text. This is a step forward in difficulty when compared to the dataset used in our proof of concept. First, the images are not cropped. Instead we are simply given full documents with text on them so both the text detector and text recognition models will have to contribute. Then, we do not have ground truth high-resolution images on this dataset. So the only option for us is to compare the performance of the OCR system on the original images vs. on the super-resoluted images, which mimics a real-life use case of our application. Finally, the FUNSD images provide an extra layer of difficulty since they also contain hand-written text. However, this dataset still offers a narrow view of the type of text we would meet in the real-world. So, to this end, we propose an experiment using the Total Text dataset, which is another step forward in difficulty of the data and thus in deciding if our hypothesis that SISR models can enhance OCR performance is true. In the Total Text dataset we have text in real-life images such as street signs, advertisements, logos, supermarket products, text on buildings and on clothes, and many more. Additionally, this text is often warped, circular, vertical, curved, and in general it simply deviates from the usual horizontal text that we are used to.

6.6.1 Shortcomings of Google's Tesseract OCR

Before running any SISR experiments on this dataset we first checked if the OCR system we are using produces acceptable results at least visually speaking. After just a few examples we can see that at least Google's Tesseract OCR system is not suitable for the Total Text dataset. We found that the current OCR system performs very poorly when it comes to real-world images in which we might have curved or vertical text and often with an unusual font. Figure 6.11 exemplifies a few such cases.

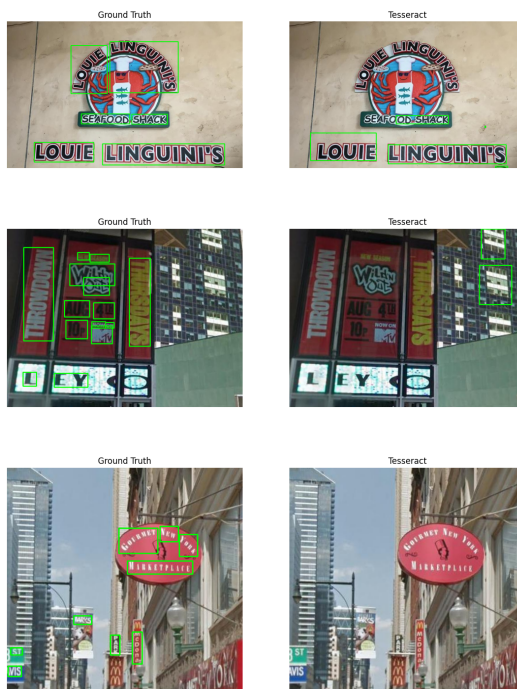


Figure 6.11: Tesseract OCR performs poorly on real-life text

Running our SISR experiment with this less than satisfactory OCR model would produce unreliable results, therefore we must find an alternative. The better option which we decided to go through with is using OpenMM Lab’s MMOCR system[32]. With this system we can choose any text detector and any text recognition model we want out of a large range of options. Among this list of options we can even choose architectures which were specially designed to cover highly-curved text. Eventually we decided to use the FCENET[68] text detector and the SATRN[69] text recognizer. The details of these models are beyond the scope of this discussion, but the important idea is that these models were specifically designed for real-life text and in general can handle curves and unusual fonts much better than Google’s Tesseract OCR. We can see in figure 6.12 how although the outputs provided by the MMOCR system are still far from perfect, they are easily better than Tesseract OCR’s outputs.

It is worth noting that this problem was not apparent up until we got to this dataset since the images from ICDAR-2015 and the ones from the FUNSD dataset were easily detectable by our previous OCR system. In the former, as we already said, the images were already cropped and the text was easily visible. In the latter, we had texts from scanned documents where the words were almost perfectly vertical everywhere and with the exception of handwritten text (where the performance degraded significantly) the font was identical everywhere.



Figure 6.12: MMOCR performs better on real-life text than Tesseract OCR

6.6.2 Original vs. SISR

Multiple experiments were conducted comparing the performance of our OCR system with the super-resolved images. Different combinations of text detection and text recognition models were tried (for text detection: FCENet[68], TextSnake[70], DBNetpp[71]; for text recognition: SATRN[69], ASTER[72], SVTR[73]). In every tried combination the processing time increased significantly (ranging from 60x to 80x), but we **have not found SISR to bring any performance gain in the OCR task**. The performance of the OCR system when applying the SISR step is either the same as the performance on the original images or ranges slightly above or below this metric (with about 1% in our average text similarity metric). There are multiple possible reasons for this. These include:

1. **The images from the Total Text Dataset already have a high resolution.** Although this dataset contains real-world images they are not of low quality. In this case a SISR model would not provide the OCR system with any additional information. The high-quality, high-resolution images already present in the dataset already have the information needed for an OCR task. This point is especially worth noting since the Total Text Dataset has not been designed for anything like what we are trying to do in this work. It was designed for creating OCR systems which are better able at detecting and recognizing highly

curved text. Because of this, the authors of the dataset were careful in obtaining already good images since the assumption was that researchers would work directly with the provided images, without any image pre-processing. In this situation the super-resolution step would just slow down processing.

2. **The text from the dataset is oftentimes either oriented vertically or significantly curved in a lot of the cases.** As already stated in the previous point, this dataset was specially designed for creating OCR systems which can tackle highly curved text. OCR systems are notoriously bad at detecting and recognizing vertical or curved text. Since this is the case, our OCR system cannot improve its performance because of the limitation posed by the orientation of the text, and an increase in image resolution would not solve this fundamental problem of the OCR systems. If an OCR system is generally poor with curved text, it will be bad even if working with high-resolution curved text.
3. **The real-world text from the dataset has highly varied fonts.** As with the previous point, OCR systems are generally poor with irregular fonts and increasing the resolution of a text with an unusual font would not address the actual problem.

6.7 Low Resolution Scanned Text Dataset

The Low Resolution Scanned Text Dataset for OCR[74] was designed and used by its authors for a similar task to what we are studying. It consists of document texts coming from different books and with varying fonts. The authors tried to perform OCR on low-resolution images by working on the other variable in our pipeline: they modified the OCR system to accommodate this need. They started from Google's standard Tesseract OCR system and applied various changes which in the end proved to be beneficial in detecting and recognizing text at low resolutions (e.g., 60-75 dpi as opposed to the standard 300 dpi images). We will use their dataset to see if we can match their performance gains by keeping this time the OCR system static (like the authors, we will take Google's standard Tesseract OCR model and we will not modify it in any way) and add our super-resolution pre-processing step. In order to be able to compare our results to the results in the paper, we will use the evaluation methods that the authors used in their work, since it is available open-source. All the needed pieces have already been developed in the previous experiments, we will simply verify the performance on this new dataset.

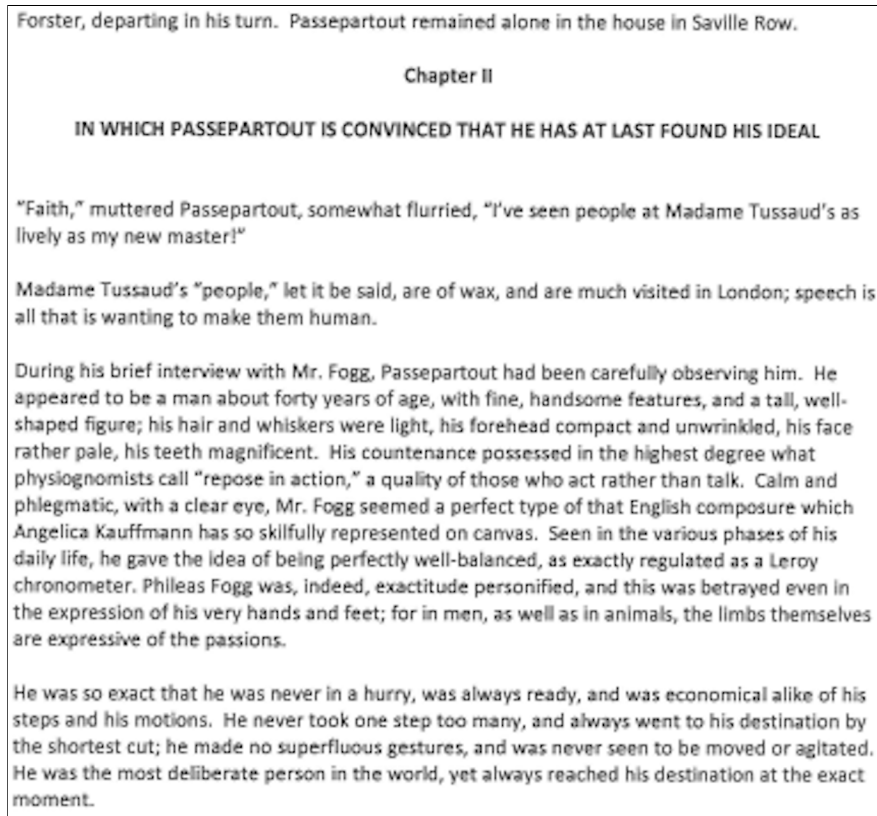


Figure 6.13: Image from the Low Resolution Scanned Text Dataset

6.7.1 The experiment

In the original paper the authors took a bunch of high resolution (300 dpi) scanned document images and artificially degraded their quality 5x (to 60dpi). Then, they compared the word level accuracy and character level accuracy that were achieved on the low resolution images upscaled with a bicubic interpolator to the original 300 dpi size vs. the same metrics which would result by using their modified Tesseract model (which implicitly upscales the images from the simulated low resolution sizes to the original 300 dpi sizes). Overall, they saw a significant performance gain: the error decreased by an average of 64% for the character level accuracy and 73% for the word level accuracy, as can be seen in table 2 of their paper[74]. It is worth noting that the authors used a 5x resolution increase for their experiments (from 60dpi to 300dpi). Because all standard super-resolution models have an upscale factor of 4x and to have an experiment as comparable as possible to that of the authors, we will upscale our images 4x in the usual way and then use bicubic interpolation to resize the images to desired final resolution which would add up to 5x total image upscaling.

Unfortunately, after carrying out our experiments we have found that **our super-resolution approach does not yield any performance gains over a simple bicubic**

interpolation technique. The results of applying Tesseract OCR on the images up-scaled with a bicubic interpolator are not exactly the same as in the paper, but they follow the general trend explored by the authors: the performance is significantly better than if we were to apply the OCR system on the original 60dpi images, but not quite as high as the performance achieved on the 300dpi images. Up until this point, the study was replicable. From this point onward however, the authors used their modified Tesseract on the low resolution images and achieved a decrease in error rate by approximately 64% on the character level accuracy and 73% on the word level accuracy over the performance achieved when using a simple bicubic interpolation, as stated in the previous paragraph. On the other hand, when using a super-resolution approach, the word and character level accuracies are indeed significantly improved when compared to the original low resolution 60dpi images, but it is worse (by a large margin) if we compare it to the simple bicubic up-sampling strategy (which is also much faster). We have observed the CLA and WLA error rates increase by an average of 220% (for example for the Helvetica font the CLA error rate with the bicubic up-sampling method is 0.72% while with super-resolution we achieved an error rate of 3.30%, resulting in an error rate increase of 223%). There are a few fonts where our super-resolution approach is better than the bicubic interpolation approach, but they are few and the performance increase is not significant (at most 30% error rate decrease). On top of the fact that our approach has a worse performance than the simple bicubic interpolation approach, the method the authors of the paper developed actually achieved a significant performance boost over the bicubic method, as already stated. So, in conclusion, **our method is significantly worse than the method developed by the authors.** On one hand, this results makes sense because the authors of the paper claimed that their approach produces better results than standard super-resolution methods. On the other hand, it is disappointing and strange to see that the super-resolution approach cannot even match the performance of the very simple bicubic interpolation method.

There are many possible reasons for which this unsatisfactory result might have been achieved. First of all, the low resolution images were then up-sampled by the authors by a factor of 5. All super-resolution models have a standard up-scaling factor of 4, so the two approach are not fully comparable. Secondly, super-resolution models are susceptible to artifacts and it is possible that under the specific conditions of this experiment our model has been especially unfortunate (60dpi images, scanned documents with various fonts, nearest neighbor down-sampling for simulating low-resolution images, etc). Then, it is possible that our combination of super-resolution + bicubic interpolation used in order to reach the up-sampling factor of 5 from this paper has produced very poor quality images.

6.8 Incidental Scene Text Dataset

The Incidental Scene Text Dataset contains real scene images which appear in the scene without the user having taken any specific prior action to cause their appearance or improve their positioning/quality in the frame. While focused scene text is the expected input for applications such as translation on demand, incidental scene text covers another wide range of applications linked to wearable cameras or massive urban captures where the capture is difficult or undesirable to control.



Figure 6.14: Example image from the Incidental Scene Text Dataset

Unfortunately, we once again **did not find the super-resolution pre-processing step to bring any performance gains**. This dataset is thus the second failed attempt at improving the OCR performance on real-life data, the first being the Total Text dataset.

6.9 Comparison with SOTA

Unfortunately, it is quite difficult to compare the results we got so far to what we can see in the literature. Even if we want to simply compare our method with the methods of others we come upon quite a few difficulties. There are quite a few studies which worked on a similar problem, meaning they tried to improve an OCR system’s performance with the help of a super-resolution model. But, out of these studies, a good portion of them have not published their datasets or even mentioned the way they created their datasets (which images they chose, what method they chose for data pre-processing, etc.).

Although old, the study Di an Ma from tried to improve OCR performance on low resolution document images. They used an outdated approach in which they multiple iterations of segmentation on pages and characters followed by K-means clustering. They found that their approach indeed improves OCR performance, but

there is no way of replicating their study or even trying to see how our method performs on their data since they used a very small dataset (a total of 5 pages of documents) and this dataset is not public or even described[75]. Similarly, Pandey et. al[76] used image super-resolution to improve OCR performance on scanned documents written in the Tamil language. They have found that their method was beneficiary, but again have not published their dataset or described how one could recreate it, so we cannot evaluate our method on their data and compare results. Once again Pandey et. al[77] worked on this problem, this time with documents in three languages: English, Kannada, and Tamil. Again, they have found that OCR accuracy improved significantly when using a super-resolution pre-processing step. But, once again, they have not published the dataset. Lat et. al [78] have also described their dataset as being composed of text from "English novels", but haven't actually mentioned which novels are these, how to recreate these datasets, or where one can find these datasets.

Out of the remaining studies, we can name a couple with which we can compare results. First, Dong et. al[79] used image super-resolution on the ICDAR-2015 competition on Text Image Super-Resolution, which is exactly what we did it in Chapter 5. In their study they have found that the super-resolution pre-processing step boost OCR performance so much that the performance on the original high-resolution images is only 1.61% higher than using the low-resolution images super-resoluted. Interestingly, we have found similar results. Using our methods, we have found that this gap between the original high-resolution images and the super-resoluted images is again very small, of only around 1%. Accounting for bigger and better models than the ones available in 2015, our results make sense and agree with the author's study.

Another study which has a published dataset is the one we worked with in section 6.7 of this chapter. However, in that study the authors did not use a super-resolution approach but actually modified the Tesseract OCR to implicitly perform a type of upscaling and thus improved their OCR performance. Unfortunately, after using their dataset of scanned documents but with our we did not find any performance gains. Possible explanations as to why this could happen were further discussed in the referenced section.

6.10 Pre-trained models vs. fine-tuning vs. training from scratch

It is important to note that throughout our experiments we have used pre-trained super-resolution models provided by the corresponding authors. All we had to do

was instantiate the model's class (which is open-source for all cases) and load the weights file which is also offered by the authors. This is the most general approach and what we actually want, since this app is meant to be as general as possible and it wouldn't be feasible to create a model with a different architecture or different weights for every conceivable use case. However, it is still important to ask ourselves and verify if we could reach better results if we were to actually train specific models ourselves. We have taken the steps needed for training a super-resolution model from scratch. We have built the model architecture, created our datasets, and set up the training procedure. Because of time and memory constrains, we had to use a limited dataset in size, the type of dataset which would have been used years ago in this subdomain (nowadays much richer datasets are used for training such algorithms). After training our Real-ESRGAN model for a few hours and a few epochs we can clearly see an improvement in our PNSR metric evaluated on the validation dataset (Figure 6.15).

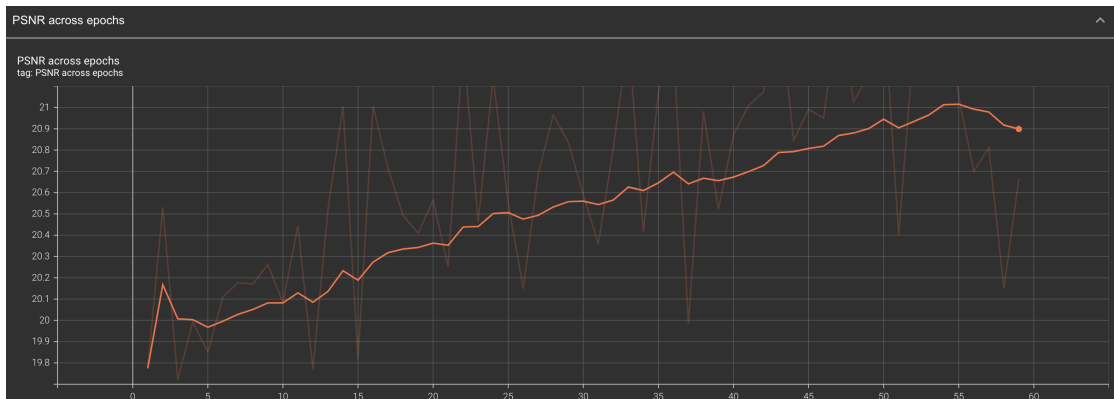


Figure 6.15: PNSR across epochs when training from scratch. Created with Tensorboard.

Furthermore, we can see that our model is actually learning to perform image reproduction to a degree even at this step, without completing the full training process. It is visually apparent that our model is slowly learning its task. We can see in Figure 6.16 how our model has started to learn to map low resolution images to their high resolution counter parts, although still imperfectly. On the left we can see the original high-resolution image patches. On the right we can see the super-resolution reconstruction image patch, which was given as input a degraded version of the high-resolution image patch from the left. There are still artifacts and blurring in the output, but this sample is taken after only a few training epochs.

These results taken during the training process look promising and indicate that we are on the right path and that our model will eventually learn accurate reconstructions provided enough training time. However, **training the model entirely from scratch is not feasible in our case**. Even these partial and imperfect results



Figure 6.16: Image reconstruction after a few epochs. Reconstructed from LR on the right. Original HR image on the left.

were achieved after a long training time (approximately 3h) and multiple runs were needed until the right hyper-parameters were chosen. This slow training process stems from the nature of these super-resolution deep learning models. Having to work with the entire image resolution throughout the model architecture and then upscale the image at the end is a very computationally expensive process, even for today's high-end GPUs. Additionally, as already stated, we had to use a much smaller dataset for our training than what is used in today's super-resolution models. This is largely because of memory constraints on our side. In order to carry out super-resolution model training the way it is done today by research labs we would need a cluster of high-end GPUs and have them available for a long time in order to run multiple experiments. This is simply not an option for the work done in this thesis.

Having ruled out the possibility of training from scratch we must ask ourselves if fine-tuning an already trained model would instead be the correct decision. This would require less computational power (so much so that a single GPU is enough) and less training time. But, there are a number of reasons for which this approach does not work either. First of all, fine-tuning only makes sense if you want to specialize your model for a specific dataset, and that is not what we want to do. We want to create a super-resolution model which would help OCR systems in general, regardless of the nature of the text the system is trying to detect and regardless of the environment of the data. Additionally, putting together a dataset for fine-tuning is either very difficult or impossible. If we were for example to fine-tune a model for the experiment we conducted with the ICDAR-2015 data, this would only **degrade**

performance because of the small dataset size. In the cases where the original training size is large and provides a good representation of real-world data and the fine-tuning dataset is small, it is known that slightly tweaking the weights of the model would actually worsen performance and you are better off to just use the trained model as is. On the other hand, if we were to fine-tune using a larger dataset such as the one from the Incidental Text Scene dataset with real-world text and scenes, this would be a very time consuming task since we would have to carry out the entire training process with image degradation and everything for potentially little to no benefit since the offered pre-trained models have already been trained on a vast range of data types, including real-world scene text or anything related to text that we might be interested in. It is therefore apparent that **fine-tuning an already pre-trained super-resolution model would bring little to no performance benefit in our task.**

With the above ideas in mind we carried out all of our experiments using already pre-trained models. It is the best option for both our computing capabilities and for our goal of creating a generalized approach of improving OCR performance.

Chapter 7

Tools and technologies

We built a simple application which offers the main functionalities of our experiments in an easy to use, user friendly manner. We have used a client-server architecture with the server being built in Python with the Flask web framework and the user interface being built with the React JavaScript framework.

Ideally, the server would be hosted on a different machine, one with a powerful GPU which could efficiently parallelize tensor computations. This is where the bulk of the work would be done. The server would have two main functions: (1) to run the super-resolution model on an image and (2) to run an OCR system on an image. Both of these functionalities require a powerful machine with plenty of computational capabilities.

7.1 Backend

7.1.1 Python

Python is a highly versatile and powerful programming language. Its simplicity and readability make it a favorite among developers, allowing them to express ideas more effectively. Python's extensive library ecosystem offers a wide range of tools and frameworks, enabling developers to tackle diverse applications with ease.

One of Python's notable strengths lies in its adaptability across various domains. It is widely used in web development, data science, and machine learning. Python's libraries, such as NumPy and pandas, simplify data manipulation and analysis, while frameworks like Django and Flask streamline web development processes. Its integration capabilities enable seamless combination with other technologies, making it a go-to choice for building complex systems.

Python's popularity is fueled by its thriving community. The active user base contributes to the creation of new libraries, tools, and resources. With online forums and extensive documentation available, developers can find support and share

knowledge easily. Python's open-source nature fosters innovation, making it a dynamic and evolving language that continues to attract developers worldwide.

7.1.2 PyTorch

PyTorch is a widely-used open-source machine learning library for Python. It provides a flexible and efficient framework for building and training deep learning models. PyTorch's main strength lies in its dynamic computational graph, which allows for easy and intuitive model development and experimentation.

With PyTorch, developers can leverage its powerful array of tools and functionalities to implement a wide range of machine learning tasks. It offers seamless integration with GPUs for accelerated computation, enabling faster training of complex models. PyTorch also provides a rich ecosystem of pre-trained models and a variety of loss functions, optimizers, and data augmentation techniques.

In our application PyTorch is used in a variety of different use cases. These include: model training, SISR/OCR inference, image manipulation, easy and parallel computing (e.g., bounding box IoU calculation), metric calculation, and many more.

7.1.3 Flask

Flask is a lightweight and flexible web framework for Python. It provides a simple yet powerful foundation for building web applications. With Flask, developers can quickly create web endpoints, handle requests, and render dynamic templates.

One of the key strengths of Flask is its minimalist design philosophy, allowing developers to choose the components they need and avoid unnecessary complexity. It provides essential features for web development, such as routing, request handling, and template rendering. Flask also supports extensions that offer additional functionality, such as database integration, authentication, and form handling, allowing developers to easily customize and scale their applications.

We implement our application's server using Flask. Here, we expose two endpoints, one for each of our main application functionalities: (1) endpoint for running super-resolution inference on an image and returning the super-resoluted image and (2) endpoint for running the OCR system and returning the bounding box polygons and their corresponding texts.

7.1.4 Matplotlib

Matplotlib is a Python plotting library known for its versatility and ease of use. It offers a wide range of plot types and customization options, allowing users to create

visually appealing and informative visualizations. With seamless integration with NumPy, Matplotlib is a popular choice for data scientists and researchers seeking high-quality plots to analyze and present data.

Matplotlib has been extensively used during our experiments and during model development. On top of this, all the visualizations from this thesis have been done in matplotlib.

7.1.5 OpenCV

OpenCV is a powerful computer vision library that provides a wide range of tools and functionalities for image and video processing in Python. Using OpenCV in Python, developers can easily load, manipulate, and analyze images and videos. It provides functions for image transformation, filtering, and enhancement, enabling tasks like resizing, cropping, and applying various image effects.

We use OpenCV for everything related to image loading/saving, encoding/decoding, resizing, and many more.

7.2 Frontend

7.2.1 React

React is a popular JavaScript framework used for building user interfaces in web applications. It enables developers to create reusable and modular UI components that update efficiently as data changes. With React, developers can easily build interactive and responsive interfaces, making it a practical choice for creating dynamic web applications.

One of the key features of React is its component-based architecture. Developers can break down their UI into smaller, self-contained components, which can be reused throughout the application. This modular approach promotes code reusability, maintainability, and scalability. React's virtual DOM efficiently updates and renders only the necessary components, resulting in faster and more optimized user interfaces.

We built the entirety of our user interface using React. On top of the basic web-site building functionalities which are provided implicitly by React, we also took advantage of React's rich ecosystem. Using libraries developed by the community we managed to implement functionalities such as image comparison slider or image mapper with relative ease.

Chapter 8

Application Implementation

8.1 Description

The proposed application allows the user to the two main features of our experiments through a user interface. These are: (1) run super-resolution on an image and (2) run OCR on an image. In the former, the client-side of our application will take an image (inputted by the user) and make an API call to the server's endpoint which is responsible for image super-resolution and will run the given image through the super-resolution model and return the super-resoluted image to the client. In the latter, the client-side of our application will again take an image as input and make an API call to the server's OCR endpoint. The server will run the given image through the OCR system and return to the client side the bounding box polygons and their corresponding texts.

8.2 Requirements and specification

8.2.1 Functionalities

The user has access to the following functionalities:

- Upload an image.
- Run one of many super-resolution algorithms on the image.
- Visualize the result of the super-resolution algorithm. This can be done in one of two ways: (1) see the original low resolution image and the super-resoluted image side-by-side or (2) overlap the two images and drag an image comparison slider to left/right to reveal different parts of the image in the low resolution version or the super-resolution version.
- Save the super-resoluted image to disk.

- Run one of many OCR systems on the image. If there is a super-resolution image available (i.e., one super-resolution model was run) that image will be used as the OCR input. If there is no super-resolved image available then the OCR system will use the original low-resolution image as input.
- Visualize the resulting bounding boxes and polygons which are outputted by the OCR system. The user can hover and click on the polygons superimposed on the image in order to get on the screen the text which was detected in that region.

8.2.2 Performance

The performance of the application depends on a number of different variables. Generally, we expect the application to run smoothly under any circumstances. However, we will mention a few of the reasons for which the application might not run optimally:

- Large super-resolution model size. Super-Resolution models work with the full size of the input image throughout the entire model architecture. For this reason, they generally require a lot of computational power to use. Even though recent architectures have made an effort to reduce the computational needs, running locally such a model is still not a feasible option.
- Large OCR system size. An OCR system has two main components: a text detection model and a text recognition model. Both of these are generally large in size and require a powerful computer if we want to use them in a reasonable time. Similarly to the above point, recent efforts to make OCR systems more light weight are still not enough and a computer with a powerful GPU is still generally needed.
- Remote server machine not powerful enough. To run the deep learning models required for our application we need a machine which can run tensor computation efficiently. For this, we need a high-end GPU which are not easy to come by. If the GPU on the server is an older/weaker one, it will significantly slow down processing time.
- Input images are too large. The deep learning models in this applications are usually fully convolutional neural networks which can work with any input image size. Because of this, the processing speed is directly correlated to the size of the input image. If you have an image which already has a high resolution and want to take that through a super-resolution algorithm and increase its size 4x one of two things can happen. Either the inference will work but

it will be slow or the memory requirements are so large that the GPU cannot accommodate them and we get a CUDA error. The same can be said about the OCR systems. The larger the input images, the slower the processing time. It is important to also note here that the maximum size of the input image we can use also depends on our GPU's memory. If we have a really high-end GPU with a low of RAM then we can fit bigger images into memory.

8.2.3 Portability

The tools and technologies used during the development of the application are cross-platform so the application can run seamlessly on any machine without depending on the system or platform of the machine.

8.3 Application design

The application is web-based. The client can interact with the application through the web user interface. Once this point is reached, the client has full access to the main functionalities of the app: super-resolution and optical character recognition. In both of the cases, the user uploads an image which is saved in a variable on the client-side. Then, when either of the two server-based functionalities are used, the client creates a connection with the server and performs a POST request to the REST server. The image is encoded and stored in the body of the request as form data. The server receives the encoded image from the form data, decodes it, and processes it according to the chosen endpoint. In the case of super-resolution, the server will encode and return the super-resoluted image. In the case of optical character recognition, the server will return two lists. One contains N pairs of 8 points each, where each pair defines a 2D polygon through 4 2D points. The other list contains N strings, where each string is the text detection corresponding to the polygon from the same index in the polygons list. Either of these possible outputs will be returned as a JSON to the client, which has been waiting until this moment. Now, the client-server connection is no longer needed and the client can proceed forward with interpreting the results from the server. In the case of super-resolution the client will simply decode the high-resolution image and display it to the user in the chosen format (side-by-side or image comparison slider). In the case of optical character recognition the client will use the returned polygons and their corresponding texts to build the image mapper and display to the user the detection polygons (along with their texts) superimposed on top of the image.

8.3.1 Server architecture

Our REST server has the purpose of exposing the two endpoints for super-resolution and optical character recognition, respectively. We present a brief documentation for these two endpoints.

Endpoint 1: Super-Resolution

- **Description:** Take as input a base64 encoded image, run super-resolution on it, and return the super-resoluted image to the requester.
- **URL:** /super-resolution
- **Method:** POST
- **Headers:**
 - **"Content-Type":** "multipart/form-data"
- **Parameters:**
 - **"image":** base64 binary encoded image
 - **"model_name":** the keyword of the super-resolution model to be used. Currently supports any one from ["esrgan", "real_esrgan", "bsrgan", "swin_ir", "swin2_sr"]
- **Request Body:** The request body will be in JSON format, where the "model_name" parameter will decide which of the available super-resolution models should be used and the "image" parameter will contain the base64 encoded image which the user wants to apply image super-resolution on. If the provided model name is not valid or the given encoded image is invalid (e.g., empty encoding string) then an error will be raised.
- **Response:** The response will be in JSON format and will contain the super-resoluted image resulted by applying super-resolution inference on the given encoded image. This super-resoluted response image will be in the "image" variable.
- **Request and response examples:**

```
POST /super-resolution
Content-Type: multipart/form-data

{
  "image": (binary),
  "model_name": "swin_ir"
}
```

Figure 8.1: SR request example

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "image": (binary)
}
```

Figure 8.2: SR response example

Endpoint 2: OCR

- **Description:** Take as input a base64 encoded image, run optical character recognition on it, and return the bounding box polygons along with their corresponding texts.
- **URL:** /ocr
- **Method:** POST
- **Headers:**
 - **"Content-Type":** "multipart/form-data"
- **Parameters:**
 - **"image":** base64 binary encoded image
 - **"model_name":** the keyword of the optical character recognition model to be used. Currently supports any one from ["tesseract", "mmocr"]
- **Request Body:** The request body will be in JSON format, where the "model_name" parameter will decide which of the available OCR models should be used and the "image" parameter will contain the base64 encoded image which the user wants to apply the OCR model on. If the provided model name is not valid or the given encoded image is invalid (e.g., empty encoding string) then an error will be raised.
- **Response:** The response will be in JSON format and will contain the bounding box polygons and their corresponding detection texts. Each bounding box polygon has a detection text associated to it. Each bounding box polygon is defined by a total of 8 integers which represent the 4 2D points which define a polygon.
- **Request and response examples:**

```
POST /ocr
Content-Type: multipart/form-data
{
  "image": (binary),
  "model_name": "tesseract"
}
```

Figure 8.3: OCR request example

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "boxes": [[65, 6, 389, 6, 389, 60, 65, 60],
            [407, 6, 718, 6, 718, 60, 407, 60]],
  "texts": ["text1",
            "text2"]
}
```

Figure 8.4: OCR response example

Also, we used the Strategy and the Factory design patterns in order to encapsulate everything related to our deep learning models (e.g., classes used, model arguments, weights file, etc). We do this to keep the logic of model selection separated

from the server-related code. Also, using these patterns, we respect the open-closed principle and improve overall code cleanliness. Doing this we can support any deep learning model for either super-resolution or OCR and adding a new one to the already built app requires minimal effort.

8.3.2 Client architecture

The client architecture of our web app focuses on the front-end components and their interaction with the server to provide a seamless user experience. The connection with the backend server is made through TypeScript async methods. The images and options selected by the user are saved in React states, which are created with `useState` hooks.

The page is divided into React components. We use a simple structure to keep our project organized: we store our images and our components in different directories.

Most of the React components we used are simple and well known such as radio buttons, image containers, or image uploader components. The more interesting components are the two components used to visualize the output of our super-resolution and OCR models. For the super-resolution output we can either visualize the output in a standard side-by-side view or, more interestingly, we can use an image comparison slider to superimpose the two images and drag a slider to see different regions of the image in either the original low resolution variant or in the high resolution variant. As for the OCR output, we use an image mapper component which will take our given 2D polygon points and draw those polygons on top of our image. Also, the texts corresponding to the polygons can easily be visualized by a simple hover or click over each drawn polygon.

The API integration with the backend server is facilitated through the `axios` library for the POST requests. We created async methods in a TypeScript file and these are used to connect to the server's endpoints.

For our styling we use a single global CSS file which provides the style for all of our components and the layout is made with the simple flexbox CSS.

We used manual testing to validate the proper functioning of our user interface. We ensured that our application does not break when uploading images of vastly different shapes and sizes. Then, we made sure that our application does not crash in any one of the following cases: attempt to perform super-resolution without any uploaded image, attempt to perform OCR without any uploaded image, attempt to use the backend functionalities while the server is stopped, attempt to download the super-resolved before actually running super-resolution on the image.

As for installation and setup, all the dependencies are installed using `npm install`

from our package.json file.

8.3.3 Use cases

The use cases of the application are exemplified in the following diagram.

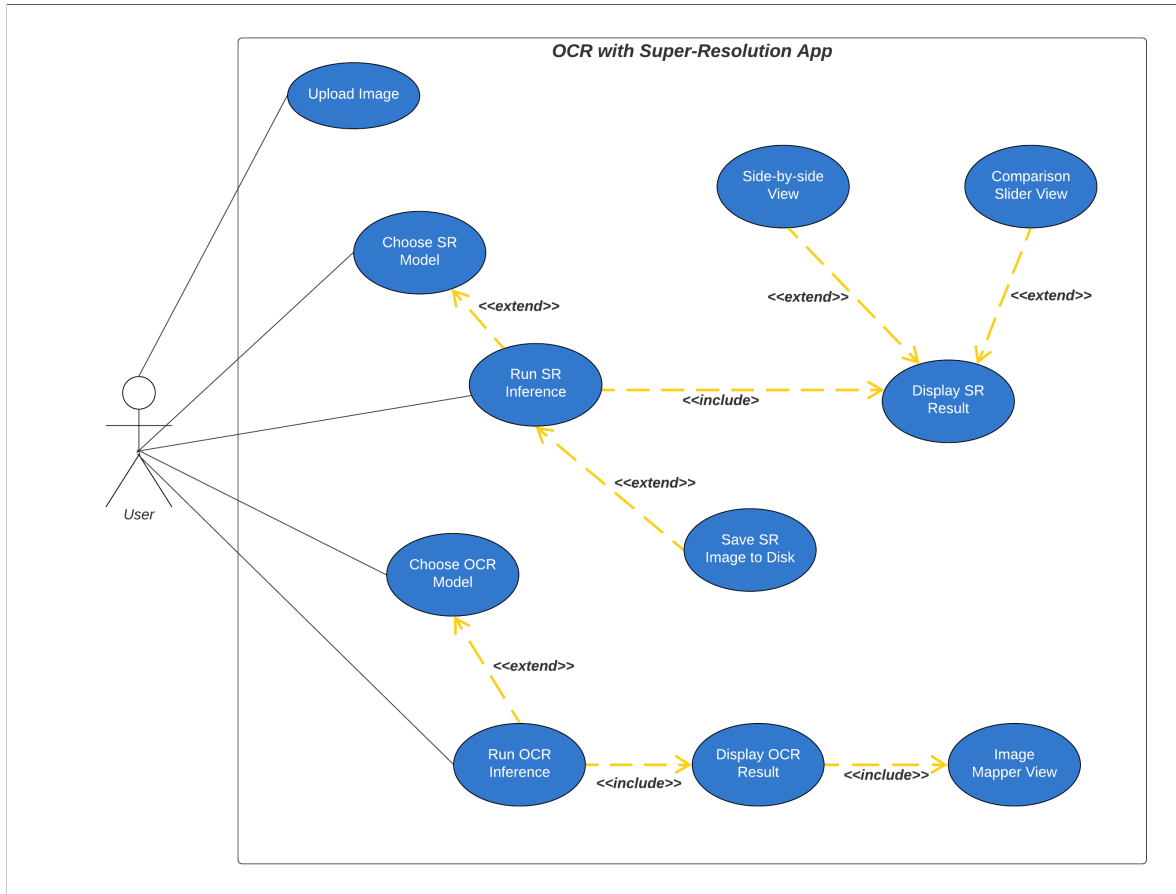


Figure 8.5: Use case diagram of our application

8.4 Application flow

As soon as the user opens the web app they can see the general outline of the website: on the left super-resolution related buttons, on the right OCR related buttons, in the upper region we have the radio buttons used for switching between the two super-resolution result view methods, in the lower region we have the two buttons responsible for image uploading and saving the super-resoluted image to disk, and finally in the center we have a large image container where we will have the uploaded image and the super-resolution result after we run it.

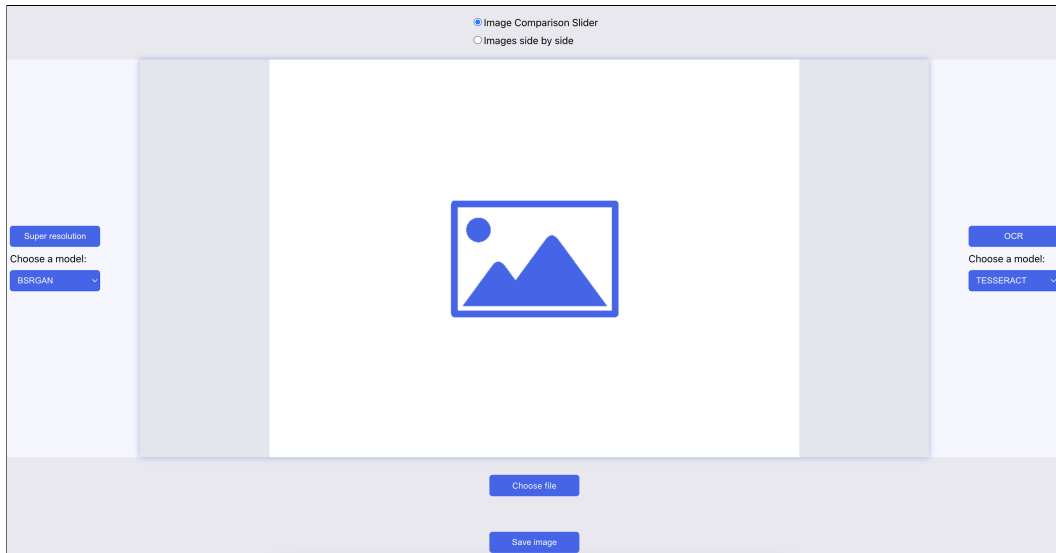


Figure 8.6: Our web app upon accessing the site

We can use the "Choose file" button to upload an image to our app and it will appear in the center of the web site. Then, using the dropdown on the left, we can choose a super-resolution model out of the list of models which have support within our app. After choosing a model, the user can press the "Super resolution". At this point the image is sent to the server which will apply the super-resolution algorithm on it. During this time, the client is waiting for the response and the user interface is in a loading screen.

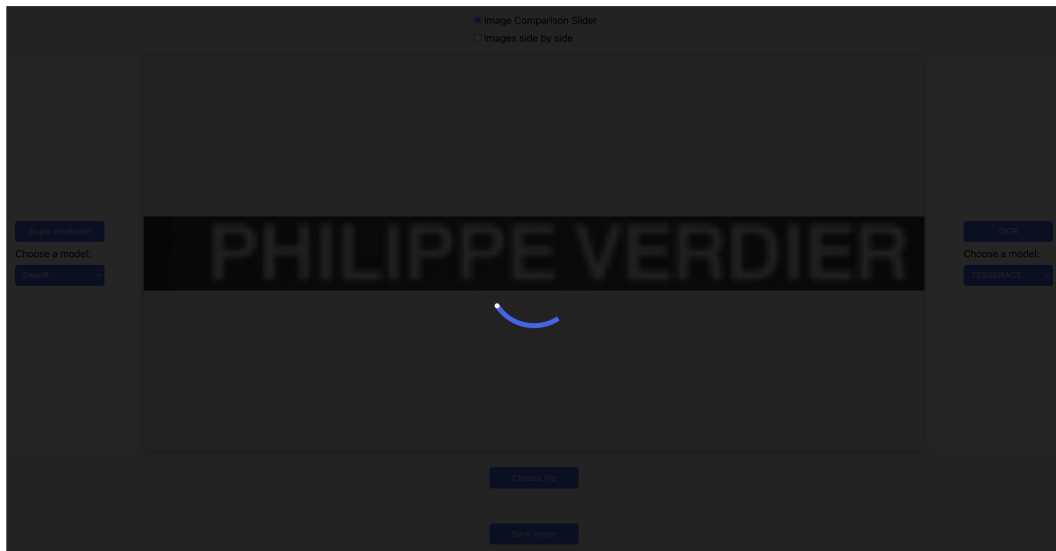


Figure 8.7: Loading screen while the server runs super-resolution inference

After the client receives the response (the super-resoluted image) from the server, it is displayed in the user interface in one of two ways. The first way is a simple

side-by-side view of the original low resolution image on the left and the processed super-resoluted image on the right. The more interesting view mode is the image comparison slider mode, where the two versions of the image are superimposed on top of each other and the user can drag a slider to the left or right to see different regions of the image in either the low resolution version or the super-resolution version.



Figure 8.8: View super-resolution result with the image comparison slider

Then, on the right side of the page the user can choose one of multiple OCR systems and then simply press the "OCR" button to run OCR inference on the super-resoluted image previously found (if the "OCR" button is used before running super-resolution then OCR will simply run on the original low-resolution image). Again, the client will send the image to the server and wait in a loading screen for the response. When the server response is received by the client, a new window will immediately appear with the super-resoluted image and the text detection polygons drawn on top of it. At this point the user can either hover on a drawn polygon and a pop-up with the corresponding text will appear and disappear when the mouse is no longer over the polygon or they can click on the polygon to lock the text pop-up on the screen until the polygon is clicked again.

Another simple functionality which the user could perform would be to save the super-resoluted image to disk, since that might come in handy and is a nice way of seeing the immediate result of our app. If such an action is desired, the user can simply press the "Save image" button at the bottom of the page at any point after the super-resolution algorithm was run.



Figure 8.9: Result of running OCR shown in an image mapper

8.5 Deployment

The server which exposes the two endpoints for super-resolution and for optical character recognition is deployed on a remote machine, and not locally. This has a number of advantages, including reducing the overhead of running heavy computation on a local machine and especially using a machine with powerful tensor calculations capabilities (i.e., a high-end GPU). Once the server is running the client has simply make API calls to the exposed endpoints and the input images will be processed much faster than anything we could achieve with a more common CPU since deep learning models do not perform well at all in such conditions.

8.6 Testing

8.6.1 Unit testing

For unit testing we chose a set of representative test images will be selected, covering a range of resolutions and image qualities. These images are passed through the image super resolution model, and the resulting enhanced images will be compared against ground truth high-resolution versions of the same images. We have ground truth images to use from datasets such as ICDAR-2015 SR competition, but another approach we used is to also run the super-resolution model on the chosen test images and save these super-resoluted images as ground truth. Then we will compare each run with these saved ground truth images. Performance metrics such as peak signal-to-noise ratio (PSNR) or structural similarity index (SSIM) are calculated to quantitatively measure the similarity between the enhanced images and

ground truth images. We do this for all of the super-resolution models we support in the application.

As for the OCR unit testing, a similar approach is used. We have taken out a set of representative test images and save their ground truth detection texts and their corresponding polygons. Then, after running different OCR systems we check that their outputs match with the saved ground truth in terms of both text detections and polygon coordinates.

8.6.2 Integration testing

Using manual testing we checked that the frontend looks decent on multiple types of images (different shapes, sizes, and extensions). We also used Chrome Developer Tools to verify that the frontend is responsive for different screen sizes. We then verified that the application doesn't crash if we press the super-resolution or OCR buttons without uploading anything, or if we press the download button without having uploaded anything/without first using super-resolution.

We used Big-Bang integration testing to verify all of this, since our application is reasonably simple and there was no need for any complex testing, at least on the client side. We also checked that the app works (i.e., we get a server response and it looks as expected) with images of different dimensions. Finally, we verified that our app doesn't break when we try to interact with the server's endpoints without the server actually being open.

Finally, we tested that our application works as a whole, again using a set of representative low-resolution test images. For each of these images we have the polygons and texts ground truth saved and after we process the images through the entire pipeline (super-resolution + optical character recognition + data transfer) we verify that the output matches our saved ground truth. All of these tests together provide a good basis for preventing any errors in future application development.

Chapter 9

Conclusions and future work

9.1 Achievements

We have successfully employed a super-resolution pre-processing step in order to study the performance of an OCR system if we upscale the input images and thus potentially improve the image's quality. We have found that under certain conditions the super-resolution pre-processing step is indeed beneficial in regards to OCR accuracy. Our study on the ICDAR-2015 Text Image Super-Resolution competition dataset has shown that OCR performance on super-resolved images starting from low resolution versions is as close as 1% to the performance achieved by that same OCR system on the original high-resolution versions. Additionally, our experiments conducted on the FUNSD dataset has also shown that OCR performance is improved on images of scanned documents.

9.2 Limitations

Unfortunately, we have not found a positive improvement of OCR accuracy in all of our experiments. We failed to see any improvement in this metric when testing our approach on real-world scene text. We have done two such experiments, one on the Total Text and one on the Incidental Scene Text datasets, any we have not seen any performance gain in either dataset.

Another important aspect is that we could not verify our approach against other results from the literature since a good deal of the published papers did not publish their datasets or even give enough details about them so that we could reconstruct these datasets and evaluate our approach in order to compare results.

Finally, we have to mention the extreme additional time required to run OCR inference on an image if we decide to use the super-resolution pre-processing step. Since we have to run each image through an entire deep learning model that is also

often times very large, we have seen our processing time increase even by as much as 60x. For some applications, this is simply not acceptable.

9.3 Future improvements

First of all, future work should be put into investigating why super-resolution does not improve the accuracy in the case of real-world scene text images. Our hypothesis is that if the image contains enough information for accurate high resolution image reconstruction then that information is also enough for accurate OCR detection and recognition and the super-resolution step would not provide any additional information. When there isn't enough information for an OCR there also isn't enough information for the super-resolution step and our SR model starts to hallucinate. This hypothesis is not confirmed or rejected by the literature since most studies on this task have focused on scanned document images and not on real-world scene text.

Another aspect which would bring great benefits would be processing time reduction. As already stated, we have experienced a significant processing time increase and in a majority of the application this increase is simply not acceptable. One could attempt to improve the speed of these super-resolution models and this would be greatly beneficial, not just for our specific use case but in any super-resolution context. For example in the case of transformer-based super-resolution models a possible improvement would be to modify the attention mechanism to use the recent Flash Attention mechanism[80]. We have not found any attempt to do this in the literature.

9.4 Conclusions

Optical Character Recognition is an important deep learning algorithm with a wide range of use cases. It is used in document digitization, text extraction from images, ID and document verification, text analytics, document translation, automatic number plate recognition, and many, many more. However, real-world images are often of poor quality, with a low resolution and/or blurry. To counteract this issue we try to pre-process our images with a super-resolution model and thus improve the quality of the image. Doing this, we have found significant OCR performance improvements for scanned document images and for the ICDAR-2015 Text Image Super-Resolution competition dataset. However, we have not found performance increases when carrying out our experiment on real-world scene text data. It is possible that whenever there is enough information for high-resolution image recon-

struction there is also enough information for OCR detection and recognition and thus this super-resolution step does not bring any additional information. Future work should focus on this aspect of real-world scene text images and on improving super-resolution processing time for the highest impact possible in this area.

Bibliography

- [1] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.
- [2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.
- [3] Robert Keys. Cubic convolution interpolation for digital image processing. *IEEE transactions on acoustics, speech, and signal processing*, 29(6):1153–1160, 1981.
- [4] Xin Li. Patch-based image interpolation: algorithms and applications. In *International Workshop on Local and Non-Local Approximation in Image Processing*, pages 1–6, 2008.
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [6] Jingyun Liang, Jiezhong Cao, Guolei Sun, Kai Zhang, Luc Van Gool, and Radu Timofte. Swinir: Image restoration using swin transformer. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1833–1844, 2021.
- [7] Marcos V Conde, Ui-Jin Choi, Maxime Burchi, and Radu Timofte. Swin2sr: Swinv2 transformer for compressed image super-resolution and restoration. In *Computer Vision–ECCV 2022 Workshops: Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part II*, pages 669–687. Springer, 2023.
- [8] James L Harris. Diffraction and resolving power. *JOSA*, 54(7):931–936, 1964.
- [9] Brian J Thompson. Modern approach to optics: Introduction to fourier optics. joseph w. goodman. mcgraw-hill, new york, 1968. xiv+ 287 pp., illus. \$13.50. mcgraw-hill physical and quantum electronics series. *Science*, 164(3876):170–170, 1969.

- [10] RY Tsai. Multiple frame image restoration and registration. *Advances in Computer Vision and Image Processing*, 1:1715–1989, 1989.
- [11] William T Freeman, Egon C Pasztor, and Owen T Carmichael. Learning low-level vision. *International journal of computer vision*, 40:25–47, 2000.
- [12] Chao Dong, Chen Change Loy, Kaiming He, and Xiaoou Tang. Image super-resolution using deep convolutional networks. *IEEE transactions on pattern analysis and machine intelligence*, 38(2):295–307, 2015.
- [13] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.
- [14] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4681–4690, 2017.
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [16] Wenzhe Shi, Jose Caballero, Ferenc Huszár, Johannes Totz, Andrew P Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1874–1883, 2016.
- [17] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part II 14*, pages 694–711. Springer, 2016.
- [18] Xintao Wang, Ke Yu, Shixiang Wu, Jinjin Gu, Yihao Liu, Chao Dong, Yu Qiao, and Chen Change Loy. Esrgan: Enhanced super-resolution generative adversarial networks. In *Proceedings of the European conference on computer vision (ECCV) workshops*, pages 0–0, 2018.
- [19] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.

- [20] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
- [21] Alexia Jolicoeur-Martineau. The relativistic discriminator: a key element missing from standard gan. *arXiv preprint arXiv:1807.00734*, 2018.
- [22] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning internal representations by error propagation. Technical report, California Univ San Diego La Jolla Inst for Cognitive Science, 1985.
- [23] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [24] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [25] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [26] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.
- [27] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, et al. Swin transformer v2: Scaling up capacity and resolution. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12009–12019, 2022.
- [28] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.
- [29] Xingkui Zhu, Shuchang Lyu, Xu Wang, and Qi Zhao. Tph-yolov5: Improved yolov5 based on transformer prediction head for object detection on drone-captured scenarios. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2778–2788, 2021.

- [30] Xinyu Zhou, Cong Yao, He Wen, Yuzhi Wang, Shuchang Zhou, Weiran He, and Jiajun Liang. East: an efficient and accurate scene text detector. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 5551–5560, 2017.
- [31] Ray Smith. An overview of the tesseract ocr engine. In *Ninth international conference on document analysis and recognition (ICDAR 2007)*, volume 2, pages 629–633. IEEE, 2007.
- [32] Zhanghui Kuang, Hongbin Sun, Zhizhong Li, Xiaoyu Yue, Tsui Hin Lin, Jianyong Chen, Huaqiang Wei, Yiqin Zhu, Tong Gao, Wenwei Zhang, et al. Mmocr: a comprehensive toolbox for text detection, recognition and understanding. In *Proceedings of the 29th ACM International Conference on Multimedia*, pages 3791–3794, 2021.
- [33] Vivien Robert and Hugues Talbot. Does super-resolution improve ocr performance in the real world? a case study on images of receipts. In *2020 IEEE International Conference on Image Processing (ICIP)*, pages 548–552. IEEE, 2020.
- [34] Ankit Lat and CV Jawahar. Enhancing ocr accuracy with super resolution. In *2018 24th International Conference on Pattern Recognition (ICPR)*, pages 3162–3167. IEEE, 2018.
- [35] Chao Dong, Ximei Zhu, Yubin Deng, Chen Change Loy, and Yu Qiao. Boosting optical character recognition: A super-resolution approach. *arXiv preprint arXiv:1506.02211*, 2015.
- [36] Rini Wongso, Ferdinand Ariandy Luwinda, et al. Evaluation of deep super resolution methods for textual images. *Procedia Computer Science*, 135:331–337, 2018.
- [37] Mikuláš Bankovic, Vít Novotný, and Petr Sojka. Application of super-resolution models in optical character recognition of czech medieval texts. In *RASLAN*, pages 11–18, 2021.
- [38] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115:211–252, 2015.
- [39] Eirikur Agustsson and Radu Timofte. Ntire 2017 challenge on single image super-resolution: Dataset and study. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 126–135, 2017.

- [40] Radu Timofte, Eirikur Agustsson, Luc Van Gool, Ming-Hsuan Yang, and Lei Zhang. Ntire 2017 challenge on single image super-resolution: Methods and results. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 114–125, 2017.
- [41] Xintao Wang, Ke Yu, Chao Dong, and Chen Change Loy. Recovering realistic texture in image super-resolution by deep spatial feature transform. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 606–615, 2018.
- [42] Kede Ma, Zhengfang Duanmu, Qingbo Wu, Zhou Wang, Hongwei Yong, Hongliang Li, and Lei Zhang. Waterloo exploration database: New challenges for image quality assessment models. *IEEE Transactions on Image Processing*, 26(2):1004–1016, 2016.
- [43] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4401–4410, 2019.
- [44] Yusuke Matsui, Kota Ito, Yuji Aramaki, Azuma Fujimoto, Toru Ogawa, Toshihiko Yamasaki, and Kiyoharu Aizawa. Sketch-based manga retrieval using manga109 dataset. *Multimedia Tools and Applications*, 76:21811–21838, 2017.
- [45] Xiaoxue Chen, Lianwen Jin, Yuanzhi Zhu, Canjie Luo, and Tianwei Wang. Text recognition in the wild: A survey. *ACM Computing Surveys (CSUR)*, 54(2):1–35, 2021.
- [46] David Martin, Charless Fowlkes, Doron Tal, and Jitendra Malik. A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In *Proceedings Eighth IEEE International Conference on Computer Vision. ICCV 2001*, volume 2, pages 416–423. IEEE, 2001.
- [47] Marco Bevilacqua, Aline Roumy, Christine Guillemot, and Marie Line Alberi-Morel. Low-complexity single-image super-resolution based on nonnegative neighbor embedding. 2012.
- [48] Roman Zeyde, Michael Elad, and Matan Protter. On single image scale-up using sparse-representations. In *Curves and Surfaces: 7th International Conference, Avignon, France, June 24-30, 2010, Revised Selected Papers 7*, pages 711–730. Springer, 2012.

- [49] Yochai Blau, Roey Mechrez, Radu Timofte, Tomer Michaeli, and Lihi Zelnik-Manor. The 2018 pirm challenge on perceptual image super-resolution. In *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*, pages 0–0, 2018.
- [50] Jia-Bin Huang, Abhishek Singh, and Narendra Ahuja. Single image super-resolution from transformed self-exemplars. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5197–5206, 2015.
- [51] Rich Franzen. Kodak lossless true color image suite. source: <http://r0k.us/graphics/kodak>, 4(2), 1999.
- [52] Lei Zhang, Xiaolin Wu, Antoni Buades, and Xin Li. Color demosaicking by local directional interpolation and nonlocal adaptive thresholding. *Journal of Electronic imaging*, 20(2):023016–023016, 2011.
- [53] Alessandro Foi, Vladimir Katkovnik, and Karen Egiazarian. Pointwise shape-adaptive dct for high-quality denoising and deblocking of grayscale and color images. *IEEE transactions on image processing*, 16(5):1395–1411, 2007.
- [54] HR Sheikh. Live image quality assessment database release 2. <http://live.ece.utexas.edu/research/quality>, 2005.
- [55] Chang Chen, Zhiwei Xiong, Xinmei Tian, Zheng-Jun Zha, and Feng Wu. Camera lens super-resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1652–1660, 2019.
- [56] Pengxu Wei, Ziwei Xie, Hannan Lu, Zongyuan Zhan, Qixiang Ye, Wangmeng Zuo, and Liang Lin. Component divide-and-conquer for real-world image super-resolution. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part VIII 16*, pages 101–117. Springer, 2020.
- [57] Jianrui Cai, Hui Zeng, Hongwei Yong, Zisheng Cao, and Lei Zhang. Toward real-world single image super-resolution: A new benchmark and a new model. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3086–3095, 2019.
- [58] Kai Zhang, Jingyun Liang, Luc Van Gool, and Radu Timofte. Designing a practical degradation model for deep blind image super-resolution. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4791–4800, 2021.

- [59] Sefi Bell-Kligler, Assaf Shocher, and Michal Irani. Blind super-resolution kernel estimation using an internal-gan. *Advances in Neural Information Processing Systems*, 32, 2019.
- [60] Gernot Riegler, Samuel Schuler, Matthias Ruther, and Horst Bischof. Conditioned regression models for non-blind single image super-resolution. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 522–530, 2015.
- [61] Kai Zhang, Wangmeng Zuo, and Lei Zhang. Learning a single convolutional super-resolution network for multiple degradations. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3262–3271, 2018.
- [62] Xintao Wang, Liangbin Xie, Chao Dong, and Ying Shan. Real-esrgan: Training real-world blind super-resolution with pure synthetic data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1905–1914, 2021.
- [63] Clément Peyrard, Moez Baccouche, Franck Mamalet, and Christophe Garcia. Icdar2015 competition on text image super-resolution. In *2015 13th International Conference on Document Analysis and Recognition (ICDAR)*, pages 1201–1205. IEEE, 2015.
- [64] Jean-Philippe Thiran Guillaume Jaume, Hazim Kemal Ekenel. Funsd: A dataset for form understanding in noisy scanned documents. In *Accepted to ICDAR-OST*, 2019.
- [65] Tomas Hrycej. *Modular learning in neural networks: a modularized approach to neural network classification*. John Wiley & Sons, Inc., 1992.
- [66] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015.
- [67] Chee Kheng Chng and Chee Seng Chan. Total-text: A comprehensive dataset for scene text detection and recognition, 2017.
- [68] Yiqin Zhu, Jianyong Chen, Lingyu Liang, Zhanghui Kuang, Lianwen Jin, and Wayne Zhang. Fourier contour embedding for arbitrary-shaped text detection, 2021.
- [69] Junyeop Lee, Sungrae Park, Jeonghun Baek, Seong Joon Oh, Seonghyeon Kim, and Hwalsuk Lee. On recognizing texts of arbitrary shapes with 2d self-attention, 2019.

- [70] Shangbang Long, Jiaqiang Ruan, Wenjie Zhang, Xin He, Wenhao Wu, and Cong Yao. Textsnake: A flexible representation for detecting text of arbitrary shapes, 2020.
- [71] Minghui Liao, Zhisheng Zou, Zhaoyi Wan, Cong Yao, and Xiang Bai. Real-time scene text detection with differentiable binarization and adaptive scale fusion, 2022.
- [72] Baoguang Shi, Mingkun Yang, Xinggang Wang, Pengyuan Lyu, Cong Yao, and Xiang Bai. Aster: An attentional scene text recognizer with flexible rectification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(9):2035–2048, 2019.
- [73] Yongkun Du, Zhineng Chen, Caiyan Jia, Xiaoting Yin, Tianlun Zheng, Chenxia Li, Yuning Du, and Yu-Gang Jiang. Svtr: Scene text recognition with a single visual model, 2022.
- [74] Julian Gilbey and Carola-Bibiane Schönlieb. Low resolution scanned text dataset for optical character recognition, July 2020.
- [75] Di Ma and Gady Agam. A super resolution framework for low resolution document image OCR. In Richard Zanibbi and Bertrand Coasnon, editors, *Document Recognition and Retrieval XX*, volume 8658 of *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, page 86580P, January 2013.
- [76] Ram Krishna Pandey, Shishira R Maiya, and A G Ramakrishnan. A new approach for upscaling document images for improving their quality. In *2017 14th IEEE India Council International Conference (INDICON)*, pages 1–6, 2017.
- [77] Ram Krishna Pandey and A. G. Ramakrishnan. Efficient document-image super-resolution using convolutional neural network. *Sādhanā*, 43, 2018.
- [78] Ankit Lat and C. V. Jawahar. Enhancing ocr accuracy with super resolution. In *2018 24th International Conference on Pattern Recognition (ICPR)*, pages 3162–3167, 2018.
- [79] Chao Dong, Ximei Zhu, Yubin Deng, Chen Change Loy, and Yu Qiao. Boosting optical character recognition: A super-resolution approach, 2015.
- [80] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.